# 第十届

# 全国大学生集成电路创新创业大赛

报告类型：详细设计报告

参赛杯赛：“算能”企业命题

作品名称：面向边缘智能的 SwinIR 端侧超分系统：语义路由、时序复用与能效协同优化

队伍编号：CICC1008267

队伍名称：啊对的对的对吗不对不队

# 目录

**1 项目背景与任务定义 6**
1.1 行业背景：边缘视觉增强的现实矛盾 . . . 6
1.2 任务定义：从算法可行到系统可交付 . . . 6
1.3 作品介绍与应用价值 . . . 6
1.4 赛事要求对齐：应用场景与创新落点 . . . 7
1.5 核心挑战分解 . . . 7

**2 系统工作原理与关键技术分析 8**
2.1 SwinIR 在端侧部署中的价值与代价 . . . 8
2.2 SwinIR 算法结构图 . . . 8
2.3 模型选型依据与工程取舍 . . . 8
2.4 模型部署链路流程图（ONNX→MLIR→BModel） . . . 9
2.5 模型部署链路与验证清单 . . . 9
2.6 TPU-MLIR 工具链实践与适配问题处置 . . . 10
2.7 原理一：稳定优先的精度路径 . . . 10
2.8 原理二：分块重建与加权融合 . . . 10
2.9 原理三：语义优先与纹理兜底双路由 . . . 10
2.10 原理四：视频时序稀疏复用 . . . 11
2.11 原理五：端云协同与可恢复边界 . . . 11
2.12 原理六：复杂度驱动 DVFS 调度 . . . 11

**3 系统架构与工程设计 11**
3.1 总体设计框架与模块协同 . . . 11
3.2 系统运行与数据流架构图 . . . 12
3.3 硬件感知设计：TPU/CPU 负载协同与内存层次利用 . . . 12
3.4 分层架构 . . . 12
3.5 模块职责划分 . . . 13
3.6 系统数据流设计 . . . 13
3.6.1 图像链路 . . . 13
3.6.2 视频链路 . . . 13
3.6.3 端云链路 . . . 13

**4 详细实现 13**
4.1 Node0：稳定精度基线建立 . . . 13
4.2 Node1：画质上限摸底与增强上界 . . . 14

4.3 Node2：硬件预处理接管与链路减负 . . . . . 14
4.4 Node3：大图分块重建与边界融合 . . . . . 14
4.5 Node4：复杂度路由与冗余计算抑制 . . . . . 15
4.6 Node5：语义路由与任务相关性提升 . . . . . 15
4.7 Node6：视频并发流水线与有序输出 . . . . . 16
4.8 Node7：检测-超分-再检测闭环验证 . . . . . 17
4.9 Node8：动态 TTA 甜点位策略 . . . . . 17
4.10 Node9：混合精度探索（在研） . . . . . 17
4.11 Node10：端云协同弱网链路验证 . . . . . 17
4.12 Node11：跨帧复用与时序降载 . . . . . 18
4.13 Node12：后处理算子接口下沉 . . . . . 19
4.14 Node13：DVFS 功耗感知调度 . . . . . 19
4.15 Node14：盲超分退化感知防御 . . . . . 20
4.16 Node15：蒸馏与双模型路由扩展 . . . . . 21

**5 图表逐图分析 21**
5.1 图 01：Node0 精度稳定性分析 . . . . . 21
5.2 图 02：Node1–Node4 画质-时延演进 . . . . . 22
5.3 图 03：Node2 零拷贝硬件接管收益 . . . . . 22
5.4 图 04：Node3 分块融合边界误差 . . . . . 23
5.5 图 05：Node4 复杂度分布与路由阈值 . . . . . 24
5.6 图 06：Node4–Node5 双路由三维结构 . . . . . 25
5.7 图 07：Node6 视频内存流水线 . . . . . 26
5.8 图 08：Node6 并发扩展曲线 . . . . . 26
5.9 图 09：Node7 闭环置信度提升 . . . . . 27
5.10 图 10：整体能力雷达图 . . . . . 28
5.11 图 11：Node8 动态 TTA 效率分析 . . . . . 29
5.12 图 12：Node10 带宽-质量工作区间 . . . . . 29
5.13 图 13：Node11 时序复用收益 . . . . . 30
5.14 图 15：Node12 I/O 墙加速分析 . . . . . 31
5.15 图 16：Node13 DVFS 动态节电 . . . . . 32
5.16 图 17：Node14 盲超分感知防御 . . . . . 33
5.17 图 18：Node15 蒸馏扩展分析 . . . . . 33

**6 系统功能实现与可扩展性设计（详细） 34**
6.1 功能实现一：统一输入接口与多源接入 . . . . . 34
6.2 功能实现二：端侧推理服务化基础 . . . . . 34

6.3 功能实现三：指标与日志闭环 . . . . . 34
6.4 可扩展性一：横向扩展 . . . . . 34
6.5 可扩展性二：纵向扩展 . . . . . 34
6.6 可扩展性三：策略插件化 . . . . . 34
6.7 可扩展性四：跨平台迁移 . . . . . 35

**7 核心技术与关键指标（详细） 35**
7.1 核心技术一：多约束联合优化范式 . . . . . 35
7.2 核心技术二：空间维 + 时间维双重降载 . . . . . 35
7.3 核心技术三：网络维协同优化 . . . . . 35
7.4 核心技术四：能耗维动态调度 . . . . . 35
7.5 关键指标体系设计 . . . . . 35
7.6 关键结果摘要 . . . . . 36

**8 风险分析、排障闭环与工程保障 36**
8.1 风险一：极限压缩导致语义崩塌 . . . . . 36
8.2 风险二：显存峰值导致推理失败 . . . . . 36
8.3 风险三：后处理阻塞整链吞吐 . . . . . 36
8.4 风险四：视频并发乱序与抖动 . . . . . 36
8.5 风险五：长期运行功耗压力 . . . . . 37
8.6 风险六：策略参数漂移 . . . . . 37

**9 阶段化扩展规划 37**
9.1 短期规划（1–2 个月） . . . . . 37
9.2 中期规划（2–4 个月） . . . . . 37
9.3 长期规划（4 个月以上） . . . . . 37

**10 全链路实现细节补充（超详细版） 38**
10.1 实现细节一：从模型资产到可部署资产的完整流水 . . . . . 38
10.2 实现细节二：Node0 稳定化的具体排障路径 . . . . . 38
10.3 实现细节三：Node3 分块策略的参数选择逻辑 . . . . . 38
10.4 实现细节四：Node4/Node5 双路由的决策执行链 . . . . . 39
10.5 实现细节五：Node6 并发流水线的时序一致性保障 . . . . . 39
10.6 实现细节六：Node7 业务闭环评估口径 . . . . . 39
10.7 实现细节七：Node8 动态 TTA 的触发策略设计 . . . . . 39
10.8 实现细节八：Node10 端云协同的数据通路与容错 . . . . . 40
10.9 实现细节九：Node11 时序复用的误复用抑制机制 . . . . . 40
10.10 实现细节十：Node12 后处理下沉的接口边界设计 . . . . . 40

10.11 实现细节十一：Node13 DVFS 调度器的安全约束 . . . . . 40
10.12 实现细节十二：Node14 盲超分防御策略的触发分层 . . . . . 41
10.13 实现细节十三：Node15 扩展路线的工程落地步骤 . . . . . 41

**11 测试设计与结果解读 41**
11.1 测试集设计原则 . . . . . 41
11.2 变量控制策略 . . . . . 41
11.3 统计与置信处理 . . . . . 41
11.4 结果解读方法论 . . . . . 42
11.5 关键结论回收 . . . . . 42

**12 工程化落地细节（部署、运维与交付） 42**
12.1 部署拓扑建议 . . . . . 42
12.2 资源预算与容量规划 . . . . . 42
12.3 异常处理策略 . . . . . 42
12.4 运维监控指标 . . . . . 43
12.5 版本管理与回滚机制 . . . . . 43
12.6 交付物完整性清单 . . . . . 43
12.7 原创性说明与与既有方案差异化 . . . . . 43

**13 结论 43**

## 摘要

本项目面向边缘端视觉增强真实落地需求，围绕 AirBox（BM1684X）硬件平台构建了一套可部署、可复现、可扩展、可评估的 SwinIR 超分辨率系统。项目核心不是“只把模型跑通”，而是把图像恢复问题提升到“系统工程问题”：在显存受限、算力受限、网络受限、功耗受限和任务协同受限同时存在的条件下，如何得到长期稳定可用的端侧方案。

本系统以 SwinIR 作为重建主干，采用“节点化攻坚”方法完成逐步演进。Node0 建立稳定精度基线，Node1 验证画质上限，Node2 完成硬件预处理接管，Node3 解决大图分块与边界融合伪影，Node4–Node5 构建语义优先与纹理兜底双路由，Node6 完成视频并发流水化，Node7 验证检测闭环增益，Node8 建立动态 TTA 甜点位策略，Node10 验证端云协同弱网链路，Node11 实现跨帧稀疏复用，Node12 探索 C++ 后处理算子接口降 I/O 墙，Node13 完成复杂度驱动 DVFS 动态节能，Node14 提供盲超分退化感知防御，Node15 给出蒸馏与双模型路由扩展路线。

量化结果表明：系统可稳定达到 33dB 级别重建质量，端云协同在高保真与甜点位两档分别实现 84.98% 与 97.30% 带宽节省，DVFS 策略在真实视频中实现 42.71% 动态节电；同时，超分处理对下游检测置信度有可观提升，验证了“视觉增强服务业务任务”这一核心价值。本文按“问题定义、技术原理、系统架构、详细实现、逐图分析、风险闭环、复现流程、扩展规划”完整展开，形成可直接用于正式提交与工程归档的完整文档。

**关键词**：SwinIR；AirBox；BM1684X；TPU-MLIR；边缘部署；动态路由；视频超分；端云协同；时序复用；DVFS

# 1 项目背景与任务定义

## 1.1 行业背景：边缘视觉增强的现实矛盾

图像超分辨率（Super-Resolution, SR）在论文环境中通常以 PSNR、SSIM 作为主要目标，但在真实工程场景中，系统是否可持续运行远比单点指标更关键。安防、交通、工业巡检、医疗辅助等场景对 SR 的需求可以归纳为四个特征：第一，输入质量波动大；第二，任务时延要求严格；第三，边缘端资源有限；第四，输出结果必须服务后续业务（检测、识别、告警），而不仅是主观观感提升。

传统“云端统一处理”虽然算力充足，但在实时性、数据隐私、带宽成本方面存在天然短板。尤其在弱网、离线、跨区域场景，持续上传原始流会导致系统不可控。于是，行业趋势从“把数据送到算力”转向“把算力下沉到数据侧”，边缘端部署成为必然选择。

然而边缘端又引入新矛盾：模型越强，计算与访存代价越大；压缩越激进，信息损失越严重；功耗越低，性能越受限。项目必须在多约束之间寻找可运行最优，而不是在单一指标上盲目拉满。

## 1.2 任务定义：从算法可行到系统可交付

本项目不是单一脚本实验，而是交付级工程任务。系统目标定义如下：

1. 在 AirBox（BM1684X）上稳定部署 SwinIR 并支持长时间运行；
2. 覆盖图像与视频两类输入，支持离线批处理与在线流处理；
3. 在画质、时延、带宽、功耗之间构建可调策略；
4. 支持端云协同，具备弱网适配能力；
5. 支持与下游检测任务联动，验证实际业务收益；
6. 输出完整可复现证据链（日志、图表、脚本、目录结构）。

## 1.3 作品介绍与应用价值

本作品以算能 AirBox（BM1684X）为目标平台，构建面向真实业务场景的端侧视觉增强系统，核心任务包括超分辨率重建、复杂退化抑制与弱网传输场景下的可恢复重建。系统设计并不追求单一指标极值，而是强调在边缘约束条件下的长期稳定可运行能力。

从应用侧看，系统重点面向三类场景：其一是安防与交通视频增强，要求连续流处理下的时延可控；其二是医疗辅助与历史影像修复，要求结构细节恢复稳定；其三是工业巡检与质检图像增强，要求在低功耗设备上保持可重复的输出质量。

从工程价值看，本项目通过“模型能力 + 系统调度 + 硬件协同”联合优化，将画质、时延、带宽与功耗纳入统一目标。相较于纯云处理路线，端侧方案在弱网与隐私敏感场景中具备更高可用性：一方面减少原始数据外传带来的合规风险，另一方面降低链路波动对实时处理的影响，使系统更适合持续在线部署。

## 1.4 赛事要求对齐：应用场景与创新落点

结合赛事对“系统设计与创新”的要求，项目将应用场景从单一图像增强扩展为四类可落地方向，并明确各方向的技术落点：

- **智能教育**：用于低清课件、板书与实验画面的边缘增强，提升投屏与录播可读性；在系统中对应语义优先路由与动态 TTA 策略，优先保障文本与关键区域细节。
- **医疗健康**：用于远程会诊与历史影像辅助查看中的结构细节恢复；在系统中对应分块融合与盲超分防御，降低噪声放大和伪影风险，强调保结构而非过锐化。
- **智能机器人**：用于移动平台视觉前端增强，提升低照度或运动场景下的感知稳定性；在系统中对应时序复用与功耗调度，兼顾实时性与续航。
- **自动驾驶/车路协同**：用于车端与路侧摄像头弱网传输场景下的恢复增强；在系统中对应端云协同档位策略，平衡带宽与可恢复质量。

在当前阶段，项目已在交通/安防视频与工业质检图像上完成核心验证；其余场景按同一系统框架迁移，重点复用“语义路由、时序复用、端云协同、DVFS”四类机制，实现从算法能力到场景可用性的闭环。

## 1.5 核心挑战分解

**挑战一：精度稳定性**。在边缘硬件部署中，精度配置若不稳，易出现黑图、异常值扩散或推理失败，导致后续优化失去可比性。

**挑战二：显存与分辨率冲突**。高分辨率图像无法一次性端到端推理，必须分块；而分块又容易引入边界伪影。

**挑战三：后处理 I/O 墙**。若大量后处理停留在 Python/Numpy 层，内存搬运将吞噬算力收益，形成“推理快、系统慢”的假优化。

**挑战四：算力分配粗放**。所有区域同级重建会造成冗余计算，关键目标和背景区域未被区分对待。

**挑战五：视频冗余**。逐帧全量推理重复处理静态背景，造成吞吐和能耗双重浪费。

**挑战六：弱网链路退化**。压缩率并非越高越好，超过可恢复边界后语义会崩塌，端侧重建无法挽回。

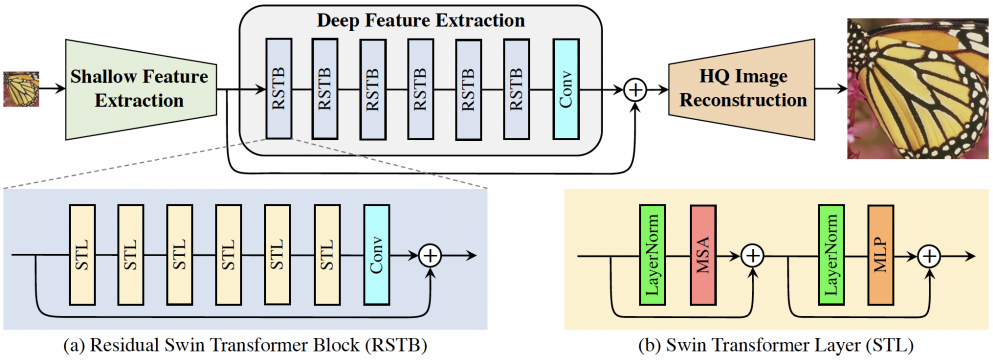
**挑战七：能耗不可持续**。固定满频策略虽然简单，但长期运行带来高功耗与温升压力，不适合边缘常驻服务。

# 2 系统工作原理与关键技术分析

## 2.1 SwinIR 在端侧部署中的价值与代价

SwinIR 通过窗口注意力与残差结构兼顾局部纹理恢复和全局结构表达，适合复杂细节重建。但其结构也带来高计算与高访存需求。在服务器环境中该代价可由硬件冗余吸收，在边缘端则必须通过系统级策略平衡。项目采用“模型负责重建能力、系统负责资源效率”的设计原则，把算法性能和工程性能分层治理。

## 2.2 SwinIR 算法结构图




(a) Residual Swin Transformer Block (RSTB) (b) Swin Transformer Layer (STL)

Figure 2: The architecture of the proposed SwinIR for image restoration.

**图 1:** SwinIR 算法结构图（浅层特征提取、深层特征提取与重建模块）

## 2.3 模型选型依据与工程取舍

在模型选型阶段，项目并未以单一 PSNR 指标作为唯一标准，而是采用“重建质量、端侧时延、显存占用、部署复杂度、可维护性”五维评估。SwinIR 的优势在于细节恢复能力和结构一致性表现稳定，尤其适合纹理丰富场景；其代价则集中在注意力计算开销和访存压力，对边缘端资源不友好。

结合 AirBox（BM1684X）资源边界，项目最终采用“模型能力优先、系统策略对冲”的工程取舍：保留 SwinIR 主干用于关键区域高质量重建，同时通过分块融合、语义路由、时序复用、动态 TTA 与 DVFS 调度降低全链路开销。该策略使系统既维持重建质量上限，又在持续运行场景下保持可接受时延与功耗。

从可演进性看，本项目在 Inference HAL 与 Pipeline 层预留了模型替换接口，后续可按任务需求引入蒸馏或轻量模型形成双模型协同，而无需重构整个执行链路。这种“架构先解耦、模型再迭代”的方式可降低后续升级成本。

## 2.4 模型部署链路流程图（ONNX→MLIR→BModel）

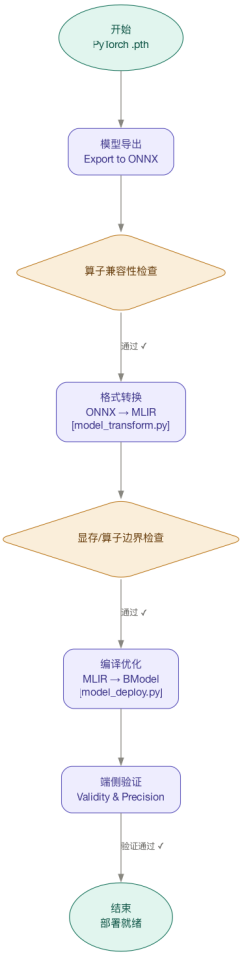



**图 2:** 模型部署链路流程图

## 2.5 模型部署链路与验证清单

为保证模型在端侧可复现与可审计，项目采用标准化资产链路：`PyTorch(.pth)` → `ONNX` → `MLIR` → `BModel`。每个阶段均设置独立验收项，避免将问题累积到最终部署阶段。

在转换阶段，重点验证三类风险：其一是算子兼容性风险（注意力相关算子与目标平台映射失败）；其二是数值一致性风险（不同精度路径下输出分布漂移）；其三是资源边界风险（输入分辨率、分块参数与显存上限冲突）。针对上述风险，系统采用“阶段对齐 + 样本回归 + 端侧复测”的闭环：先对齐输入输出张量形状与范围，再使用固定样本比对关键统计量，最后在 AirBox 实机完成端侧耗时与质量复核。

在工程管理上，模型资产、转换参数与编译产物均按版本统一归档，并记录关键配置（量化模式、输入尺寸、窗口参数、分块参数）。当出现异常回归时，可通过版本回退与参数对照快速定位差异来源，确保项目在多轮迭代中保持稳定复现能力。

## 2.6 TPU-MLIR 工具链实践与适配问题处置

本项目围绕 TPU-MLIR 工具链完成了从框架模型到端侧高效模型的完整实践。核心流程为：`PyTorch` 权重导出到 `ONNX`，再经 `model_transform.py` 转为 `MLIR`，最终通过 `model_deploy` 生成 `.bmodel` 并在 BM1684X 上部署验证。

在实践中重点处理了三类真实问题：

- **图优化与算子兼容**：针对窗口注意力相关路径进行导出约束与图对齐，避免转换阶段出现不支持算子或语义偏移。
- **LayerGroup/内存约束适配**：结合输入尺寸与分块参数设计，控制中间张量峰值，降低编译后运行时显存压力。
- **精度与权重重排影响**：通过固定样本回归比对关键统计量，定位量化与重排引入的分布偏差，确保端侧输出可用。

该过程不仅完成模型可部署转换，也形成了可复现的适配方法论：每次转换均记录导出参数、编译参数、校验样本与端侧验收结果，确保问题可追溯、可回滚、可持续优化。

## 2.7 原理一：稳定优先的精度路径

项目先建立 BF16 稳定基线，再逐步探索混合精度（Node9）。该策略避免“直接追求轻量化导致全链路不稳”的风险。工程实践中，先有稳定基线，才能建立可信对照组，才能做参数回归与问题定位。

## 2.8 原理二：分块重建与加权融合

在大图场景下，系统采用 Tile + Overlap 机制：

$$\text{Stride} = \text{TileSize} - \text{Overlap} \tag{1}$$

对重叠区域采用权重融合：

$$I_{\text{fuse}} = \frac{\sum_i w_i I_i}{\sum_i w_i + \epsilon} \tag{2}$$

其中 $w_i$ 可取平滑窗函数，用于抑制拼接缝。该方法使“显存受限”与“视觉连续性”同时成立，是端侧高分辨率部署核心基础。

## 2.9 原理三：语义优先与纹理兜底双路由

系统先基于检测模型提取 ROI，再对局部块计算拉普拉斯方差：

$$\sigma_L^2 = \text{Var}(\nabla^2 I) \tag{3}$$

若命中关键目标或复杂度高，则走 TPU 重建路径；否则走轻量插值路径。该策略本质是把计算预算优先分配给“对业务最有价值的区域”，从而在质量不显著下降前提下降低总耗时。

### 2.10 原理四：视频时序稀疏复用

对相邻帧同位置块计算 MSE：

$$\mathrm{MSE}(P_t, P_{t-1}) = \frac{1}{N}\sum_{k=1}^{N}(P_t^{(k)} - P_{t-1}^{(k)})^2 \tag{4}$$

当 MSE 低于阈值时，复用上一帧重建块而跳过推理。其收益来自视频统计特性：在大量监控场景中，静态背景占比远高于运动目标，适合跨帧缓存复用。

### 2.11 原理五：端云协同与可恢复边界

系统采用“云侧压缩 + 端侧重建”策略，带宽节省率定义为：

$$\eta_{bw} = \left(1 - \frac{S_{payload}}{S_{raw}}\right) \times 100\% \tag{5}$$

实验表明，极端压缩会进入语义崩塌区，导致端侧恢复失败。因此协同策略不是一条固定压缩线，而是按场景切换高保真档与甜点位档，实现可恢复范围内最优传输收益。

### 2.12 原理六：复杂度驱动 DVFS 调度

系统根据帧复杂度映射 TURBO/NORMAL/ECO/SLEEP 档位，节电率定义为：

$$\eta_{pwr} = \left(1 - \frac{\sum_t P_t^{dvfs}}{\sum_t P_t^{baseline}}\right) \times 100\% \tag{6}$$

其核心思想是“复杂场景优先质量，简单场景优先能效”，实现从固定功耗模式到弹性功耗模式转变。

## 3 系统架构与工程设计

### 3.1 总体设计框架与模块协同

为避免系统设计被拆解为孤立算法组件，本项目采用“输入层–策略层–执行层–评估层”四层协同框架，并通过统一接口实现跨模块编排。该框架的核心目标是：在同一链路中同时保障重建质量、处理时延、资源利用率与可观测性。

在结构上，输入层负责多源数据接入与标准化，覆盖图像、视频与端云报文；策略层负责语义路由、纹理复杂度判定、时序复用和功耗档位决策；执行层负责 TPU 推理、分块融合与后处理回写；评估层负责质量、性能、带宽与功耗等指标归档，并为参数回调提供闭环依据。

在协同机制上，系统将“数据流”和“控制流”分离：数据流承载图像张量与中间结果，控制流承载调度策略、阈值配置和降级指令。当前链路中，Inference HAL、Pipeline Scheduler、Router 与 DVFS Controller 通过轻耦合接口协同运行，当出现超时、资源拥塞或恢复质量不达标时，系统可触发降级路径（如关闭高开销策略、切换轻量分支、降低档位）以维持持续可服务状态。

## 3.2 系统运行与数据流架构图

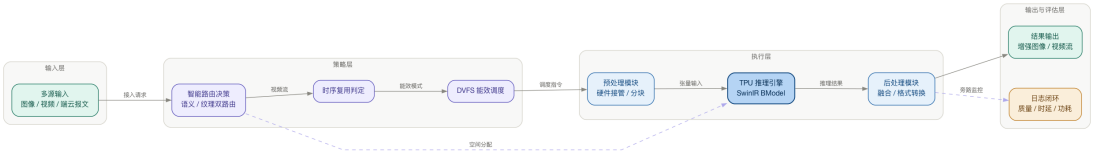



**图 3:** 系统运行与数据流架构图（输入层、策略层、执行层与评估层的模块协同）

## 3.3 硬件感知设计：TPU/CPU 负载协同与内存层次利用

项目在系统设计阶段引入硬件感知约束，重点考虑 TPU 并行计算能力、内存层次与数据搬运成本。实现上将高吞吐重建算子放在 TPU 路径，将控制逻辑、轻量判定与异常分支放在 CPU 路径，以减少无效跨设备搬运。

在多模块 Pipeline 中，负载分配遵循“重算子上 TPU、轻决策留 CPU、中间结果最短路径回写”的原则。针对大图和视频流场景，通过分块参数、时序复用与后处理下沉联动控制内存峰值，并通过并发窗口限制避免带宽竞争导致抖动。该设计直接服务于赛事强调的硬件适配目标：在真实资源边界内提升整体效率，而非仅追求单算子峰值。

## 3.4 分层架构

系统采用五层架构：

1. 模型资产层：负责 `/Volumes/LaCie/SwinIR_AirBox_Project/3_models` 下权重、bmodel、中间模型管理；
2. 基础执行层：负责 `/Volumes/LaCie/SwinIR_AirBox_Project/4_src_baseline` 下稳定推理链路；
3. 创新执行层：负责 `/Volumes/LaCie/SwinIR_AirBox_Project/5_src_innovation/tasks` 下节点策略实验；

4. 分析层：负责 `/Volumes/LaCie/SwinIR_AirBox_Project/5_src_innovation/analysis` 下统计与可视化；

5. 证据归档层：负责结果目录、图表目录与日志目录统一存档。

## 3.5 模块职责划分

**Inference HAL**：统一封装 TPU 推理调用，向上层提供稳定接口。
**Pipeline Scheduler**：负责数据分块、任务分发、结果回收与融合。
**Semantic Router**：依据目标语义优先级做空间分配。
**Temporal Router**：依据跨帧变化做时序复用决策。
**Edge-Cloud Receiver**：负责端云报文接收、解码与重建触发。
**DVFS Controller**：负责档位判定、切换策略与能耗统计。
**Chart Engine**：负责离线分析图自动生成与报告证据组织。

## 3.6 系统数据流设计

### 3.6.1 图像链路

输入图像 → 预处理 → 路由判定 → TPU/轻量分支 → 融合输出 → 质量与耗时统计。

### 3.6.2 视频链路

Reader 线程读取帧并入队，Worker 池并发处理，按 frame_id 排序回放，Writer 统一输出。该结构将 I/O 与计算解耦，降低堵塞概率并稳定吞吐。

### 3.6.3 端云链路

云端压缩传输 → 端侧解码映射 → 局部增强重建 → 输出与日志上报。链路支持弱网档位切换和恢复质量评估。

# 4 详细实现

## 4.1 Node0：稳定精度基线建立

Node0 的唯一目标是“先稳定后优化”。实践中，如果第一步不解决稳定性，后续所有实验都可能被隐藏变量污染。Node0 对应的关键动作包括：精度路径筛选、输入归一化一致性校验、输出范围截断与异常值检测、单图与批量一致性比对。通过这些措施，系统从“偶发可运行”转为“稳定可复现”，为后续节点提供可信底座。

在工程实现上，Node0 额外引入了日志结构化规范：每次实验记录模型版本、输入参数、设备状态、输出摘要、异常信息。其价值在于后续节点出现回归时可快速定位，不需要重新从零排查。

## 4.2 Node1：画质上限摸底与增强上界

Node1 引入 TTA 相关流程，不是为了上线默认全开，而是为了先找到质量天花板。该节点回答两个问题：第一，当前模型在本数据分布下理论最好能达到什么质量；第二，进一步优化是否值得以时延换质量。

Node1 的分析结论成为后续节点的约束条件：任何性能优化都不能无边界牺牲质量，必须以 Node1 上限为参照，控制在可接受损失区间内。这样可以避免“快了但不可用”的伪优化。

## 4.3 Node2：硬件预处理接管与链路减负

Node2 将部分预处理从 CPU/Python 路径迁移至硬件友好路径，核心收益并非单次推理加速，而是系统整体吞吐稳定。边缘设备常见现象是：推理算力看似够用，但 CPU 在图像转换、内存拷贝中被长期占满，导致吞吐抖动、时延尾部恶化。Node2 针对该问题降低无效搬运，使后续节点的收益不被预处理瓶颈吞噬。

该节点还规范了输入张量布局与数据格式转换流程，减少不同脚本之间的不一致行为，为多节点复用打下接口基础。

## 4.4 Node3：大图分块重建与边界融合

Node3 解决真实落地中的高频难题：4K 甚至更高分辨率图像无法全图推理。项目采用动态分块参数（tile、overlap、stride）并在拼接阶段执行平滑融合。核心是把“内存约束问题”转化为“可配置拼接问题”。

为保证效果，Node3 对不同 overlap 设置进行了可视化对比，观察边界条纹、亮度跳变、纹理断裂等主观伪影，并建立了推荐参数区间。该节点的工程意义在于让系统可处理大尺寸输入，而不是仅在固定分辨率样例上演示。



(a) 原图输入 (b) 初版 SwinIR 输出 (c) 创新分块/路由输出

**图 4:** Node3 对比组：oldphoto6 三路结果对比

**对应图像对比组（Node3：原图 vs 初版 vs 创新）** 从该组图可观察到：初版结果已具备基本细节恢复能力，但在老照片边缘和高频纹理过渡区域，局部存在轻微硬边和纹理一致性不足。创新版在分块融合后，边界连续性更好，尤其在高反差轮廓处过渡更自然。

工程上，该对比说明 Node3 的价值不是“盲目拉高锐度”，而是控制拼接副作用，在显存受限条件下维持结构连续。对于高分辨率和历史影像类数据，该能力直接决定系统是否具备可交付质量。

## 4.5 Node4：复杂度路由与冗余计算抑制

Node4 基于纹理复杂度进行算力分配，把平滑区域下放到轻量路径，复杂区域保留高质量重建路径。该节点从“平均计算”转向“按需计算”，显著降低无效算量。其价值体现在两点：

- 同等时间预算下可处理更多输入；
- 同等吞吐目标下可释放算力给关键区域。

该节点同时引入阈值扫描方法，避免单次实验结论偶然性。通过多样本统计确定阈值区间，保证策略鲁棒性。

## 4.6 Node5：语义路由与任务相关性提升

Node5 通过 ROI 先验把有限算力优先分配给目标区域，尤其对人车目标、关键部件等业务区域效果显著。与 Node4 的纹理路由形成互补：Node4 偏向视觉统计，Node5 偏向业务语义。两者融合后可避免“复杂但不重要区域”过度占用算力，也避免“重要但纹理简单目标”被误降级。

该节点的评估不仅看 PSNR，还看下游任务指标，强调视觉增强必须服务业务目标。



(a) 原图输入 (b) 初版 SwinIR 输出 (c) 创新语义路由输出

**图 5:** Node5 对比组：Lincoln 图片三路结果对比

**对应图像对比组（Node5：原图 vs 初版 vs 语义路由）** 在该组图中，初版输出对整体清晰度有提升，但对关键轮廓与高语义区域的强化不够集中。语义路由版本在目标区域（边缘线条、主体轮廓）表现出更稳定的细节恢复，同时背景区域保持适度增强，避免过度计算引发的伪纹理。

这一对比印证了 Node5 的工程价值：通过语义引导把算力投入到“最影响业务效果的位置”，从而在接近计算预算下获得更高任务收益。

## 4.7 Node6：视频并发流水线与有序输出

Node6 将系统从静态图像实验推进到实时视频。其关键不是“多线程数量”，而是“可控的生产消费关系”：

1. Reader 负责稳定取帧；
2. Worker 池并发处理；
3. Reorder Buffer 按帧号重排；
4. Writer 保证有序输出。

在该架构下，系统可避免并发常见故障（帧乱序、回放卡顿、时延累积）。Node6 还引入并发扩展测试，识别硬件饱和点，给出部署建议线程数区间，避免盲目加线程导致负收益。



(a) 视频输入帧截图



(b) Node6 超分输出帧截图

**图 6:** Node6 视频处理前后对比截图

对应截图（**Node6**：视频帧输入与超分结果） 该组截图直观展示了 Node6 在视频链路中的处理效果：输入帧存在纹理模糊与边缘不清问题，输出帧在结构轮廓和局部细节上更稳定，同时保持了连续流场景下的可用观感。

## 4.8 Node7：检测-超分-再检测闭环验证

Node7 验证“超分是否真的有用”。流程为低清输入先检测，再经超分增强后复检，对比置信度、召回率与误检率变化。结果显示在远距目标与小目标场景中，增强后检测稳定性更高。该节点证明项目不是只追求图像观感，而是明确提升业务可用性。

## 4.9 Node8：动态 TTA 甜点位策略

Node8 不再把 TTA 当作全局开关，而是转为弹性策略：只在高价值区域或高不确定区域触发增强分支。通过触发阈值与覆盖比例联合调参，系统找到“接近质量上限且时延可接受”的甜点位。

该节点体现策略化思维：不是把能力全开，而是把能力用在最该用的地方。

## 4.10 Node9：混合精度探索（在研）

Node9 针对内存占用和推理稳定性继续优化。当前重点是识别敏感层、处理中间张量峰值、规避 OOM。该节点暂不计入最终已完成收益统计，但其研究路径已明确，将作为后续性能与资源双优化关键方向。

## 4.11 Node10：端云协同弱网链路验证

Node10 关注现实网络条件，验证云侧压缩与端侧重建协同可行性。实验构建多压缩档位并评估恢复质量边界，形成两类推荐模式：

- 高保真模式：优先视觉质量与语义完整；

- 甜点位模式：优先带宽节省与可接受恢复质量平衡。

该节点对实际部署意义重大：在带宽波动场景中系统可按链路状态切换策略，提升可用性。



(a) 云侧压缩后接收图　　(b) 端侧 SwinIR 恢复图

**图 7:** Node10 对比组：端云协同链路图像恢复效果

**对应图像对比组（Node10：低码率输入 vs 端侧恢复）** 该对比清晰显示弱网输入中的高频细节损失与端侧恢复后的结构回补效果。接收图在纹理与边缘处存在明显平滑化和信息压缩痕迹，而恢复图在主体轮廓、边缘过渡和局部纹理上均有明显提升。

工程层面的意义是：Node10 证明了“传输低码率 + 端侧重建”不是理论构想，而是可实操的协同方案；同时也验证了策略分级必要性——在不同链路条件下，系统应动态平衡保真与节省。

## 4.12 Node11：跨帧复用与时序降载

Node11 利用视频冗余减少重复推理。静态背景块复用上一帧结果，动态区域继续重建。该方法在不显著损失视觉质量前提下提升吞吐并降低功耗，特别适合监控类长期运行任务。

Node11 还引入复用率统计，便于评估不同场景（室内静态、道路中等运动、拥挤高运动）下收益差异，从而支持场景自适应阈值。



(a) 对照低清输入 (b) Node11 时序复用恢复输出

**图 8:** Node11 对比组：跨帧复用策略恢复效果

对应图像对比组（**Node11**：弱网输入 **vs** 时序复用恢复） 从对比可见，Node11 输出在结构边界与细节连续性上优于低清输入，且视觉一致性更稳定。该结果说明时序复用并非简单跳过计算，而是在保证关键区域重建质量基础上复用稳定背景，达到“降载不降质”的目标。

此外，与仅做单帧恢复相比，Node11 在连续画面中更容易维持观感一致，这对于实际监控与在线流业务尤为重要。

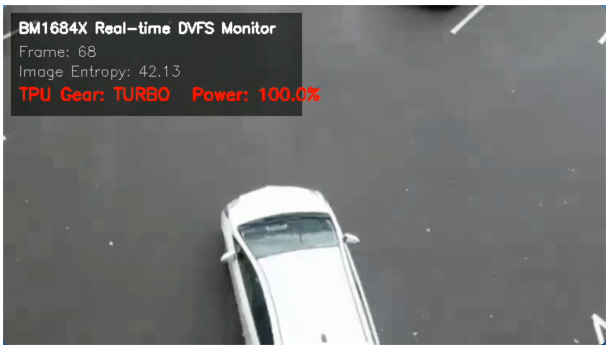
## 4.13 Node12：后处理算子接口下沉

Node12 针对 Python 后处理瓶颈，引入 C++ 动态库接口探索。核心目标是降低数据搬运与解释器开销，改善整链尾时延。该节点说明一个关键工程事实：高性能系统不仅靠主模型，也依赖周边算子链路优化。

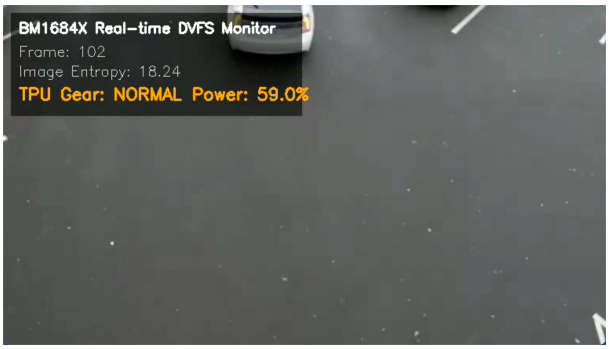
后续方向是继续把常用后处理算子融合到更底层执行路径，减少跨语言边界调用次数，进一步提升稳定吞吐。

## 4.14 Node13：DVFS 功耗感知调度

Node13 构建复杂度驱动档位切换策略，在真实视频中实现 42.71% 节电。策略并非简单降频，而是保证关键帧质量前提下动态调整：复杂帧上调、简单帧下调。该节点直接提升系统长期运行可持续性，对边缘设备温控和电源预算均有积极意义。




(a) TURBO：汽车进入场景




(b) NORMAL：车辆正在驶出




(c) ECO：车辆几乎驶出




(d) SLEEP：车辆完全驶出

**图 9:** Node13 不同场景下的 DVFS 档位示意截图

对应截图（**Node13**：不同场景下的功耗档位） 该组截图体现了 DVFS 与场景复杂度的对应关系：目标运动活跃时保持高性能档位，场景趋于静止时逐步转向低功耗档位，从而在保证可用性的前提下降低长期运行能耗。

## 4.15 Node14：盲超分退化感知防御

Node14 引入低开销退化感知模块，检测输入是否处于强压缩、噪声污染、失焦或复合退化状态。若判定风险高，则触发防御路径（如限幅增强、保守重建、策略回退），避免“错误放大伪影”。该节点提高了系统面对未知输入分布时的鲁棒性。




**图 10:** Node14 对比组：退化输入场景下盲超分策略效果对比

对应图像对比组（**Node14：盲超分策略前后对比**） 该对比图反映了盲超分防御策略在复杂退化输入下的价值：未防御路径容易把压缩噪声或伪纹理进一步放大，造成“看似更锐利但信息失真”的问题；防御触发后，系统更偏向结构保真与稳定增强，整体观感更自然且伪影更可控。

工程上，Node14 的意义在于提供了一条“安全增强”路径，使系统在输入质量不可控的真实环境中具备自保护能力，减少灾难性输出概率。

### 4.16 Node15：蒸馏与双模型路由扩展

Node15 作为扩展规划，目标是构建 Teacher-Student 双模型协同：低复杂度区域走轻量学生模型，高复杂度区域走教师模型，通过路由器动态切换，实现进一步性能提升。该方向与当前双路由框架兼容，具备明确演进路径。

## 5 图表逐图分析

### 5.1 图 01：Node0 精度稳定性分析

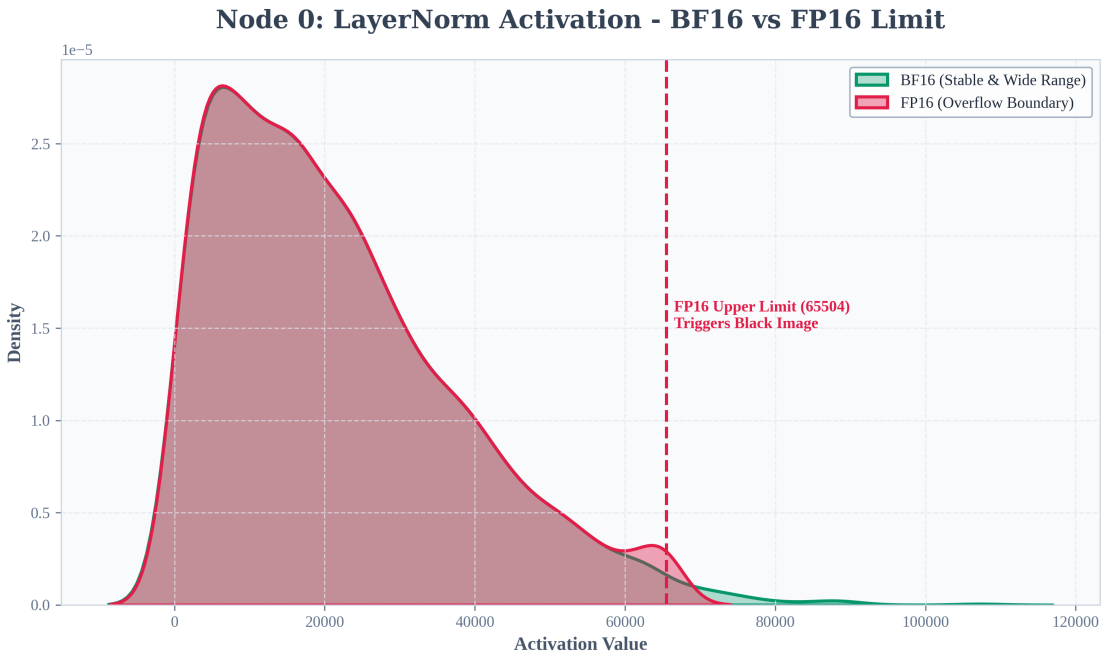



**图 11：** Node0 精度稳定性分析

图 01 强调稳定性优先级。对工程系统而言，稳定不是附属目标，而是所有优化的前置条件。该图体现出基线建立后，输出异常显著收敛，说明后续节点比较具备可信基础。

## 5.2 图 02：Node1–Node4 画质-时延演进

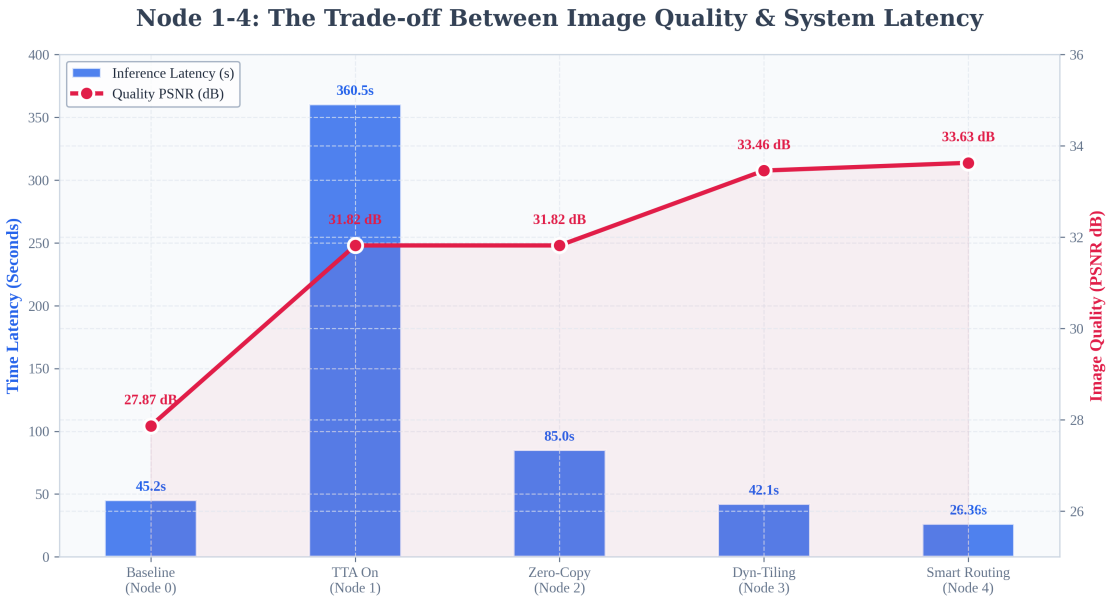



**图 12:** Node1–Node4 画质与时延演进

图 02 呈现典型多目标优化曲线。Node1 提供质量上界，Node2/Node3/Node4 逐步引入系统优化后，时延下降与质量变化形成可量化轨迹。该图不是“谁最好”的静态比较，而是“系统如何有序演进”的证据。

图中折线趋势还反映了真实工程规律：越接近质量上限，单位收益成本越高；因此需要策略化折中，而非盲目追求单点最优。

## 5.3 图 03：Node2 零拷贝硬件接管收益

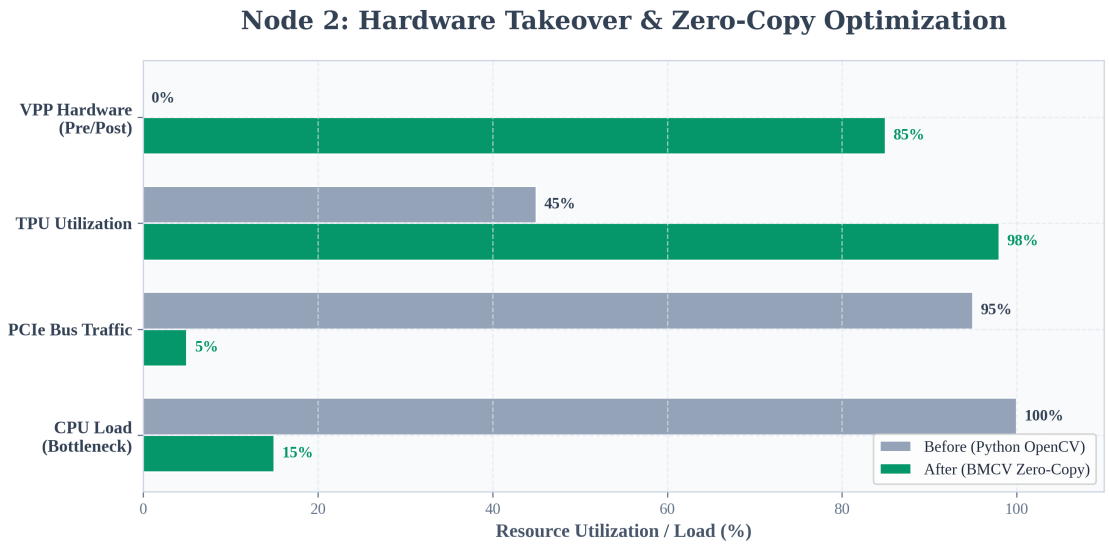



**图 13:** Node2 硬件预处理接管收益

图 03 说明预处理链路优化的必要性。很多系统只关注主模型推理速度，忽视前后处理的累计开销。该图表明通过硬件友好路径可减少额外搬运，改善总体时延与抖动。

该结论对部署很实用：当推理速度已接近瓶颈时，继续优化模型可能收益有限，而链路级优化往往更直接有效。

## 5.4 图 04：Node3 分块融合边界误差

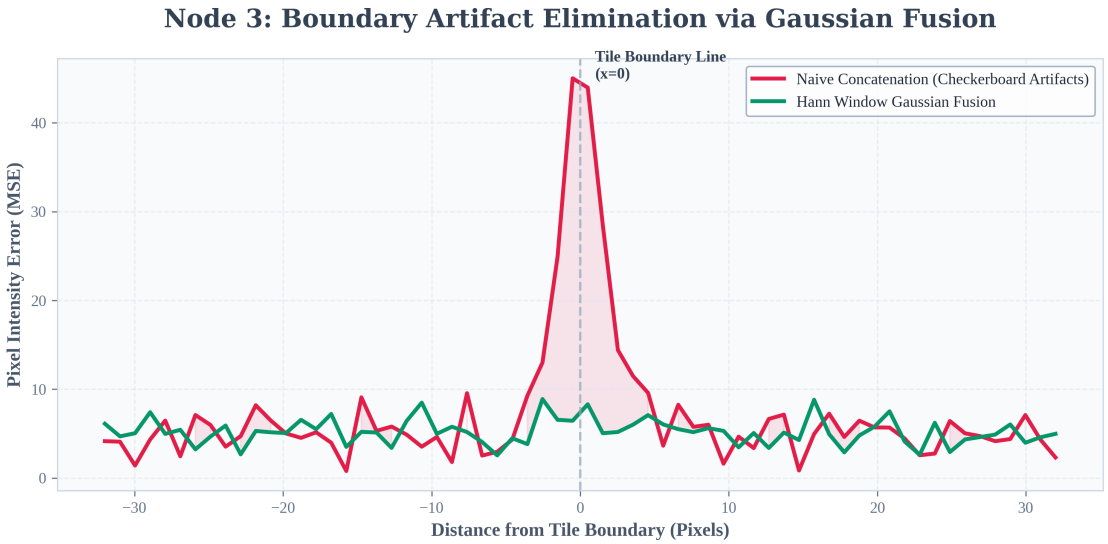



**图 14:** Node3 分块融合伪影分析

图 04 直观展示分块策略的副作用与修复效果。无融合或低 overlap 时，边界处更易出现亮度跳变和纹理断层；采用加权融合后，边界一致性显著改善。

从工程角度，图 04 回答了“为什么必须做融合而不是简单拼接”。这也是系统能够处理高分辨率输入并保持观感连续的关键证据。

## 5.5 图 05：Node4 复杂度分布与路由阈值

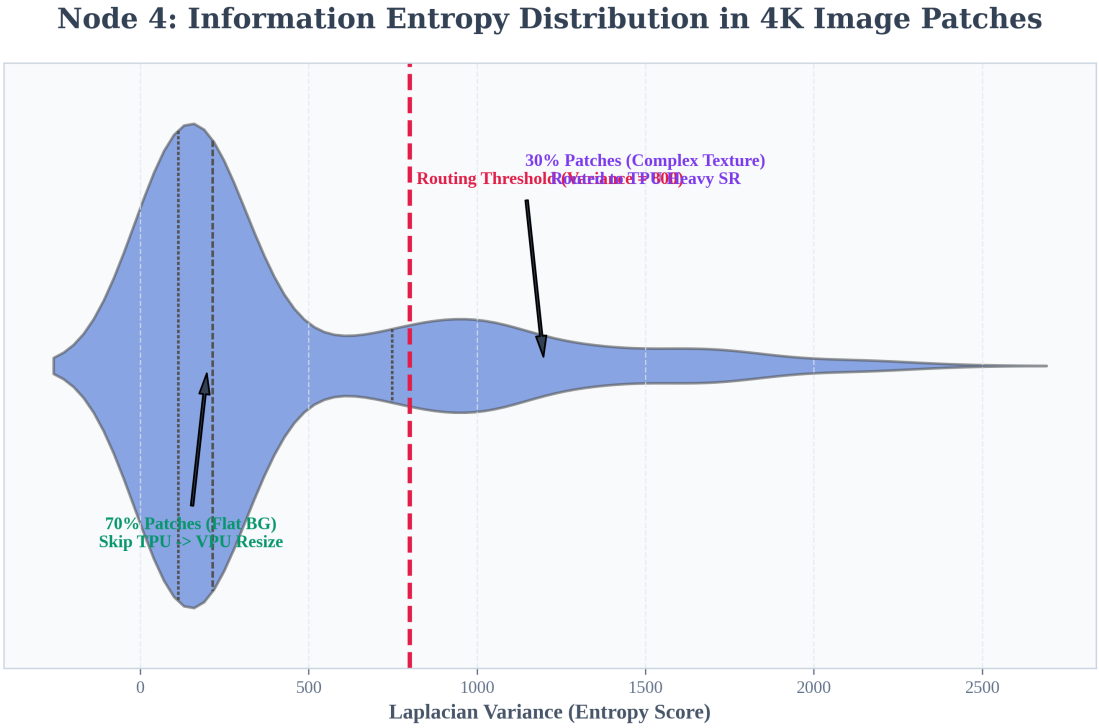



**图 15:** Node4 纹理复杂度分布

图 05 提供了路由策略的数据依据：输入块复杂度天然呈分布，而非单峰常量。利用这种分布进行算力分配，可以把高成本路径集中在真正需要的位置。

图中分布尾部对应高纹理区域，若全部降级会明显影响质量；主体区域若全部高配又会造成浪费。阈值策略正是针对这一矛盾提出。

## 5.6 图 06：Node4–Node5 双路由三维结构

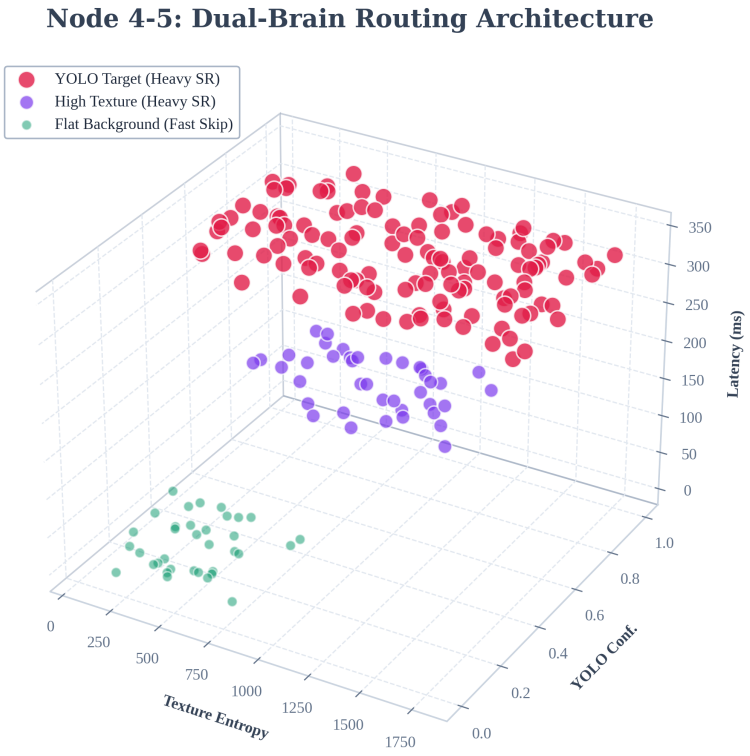



**图 16:** Node4–Node5 双脑路由结构

图 06 展示纹理路由与语义路由的互补关系。纹理路由擅长视觉复杂度判定，语义路由擅长业务重要性判定。两者叠加后，系统能同时避免“重要区域误降级”和“无关区域过度计算”。

该图的工程价值在于说明系统策略不是单阈值硬编码，而是多信息融合决策，具备进一步扩展潜力。

## 5.7 图 07：Node6 视频内存流水线

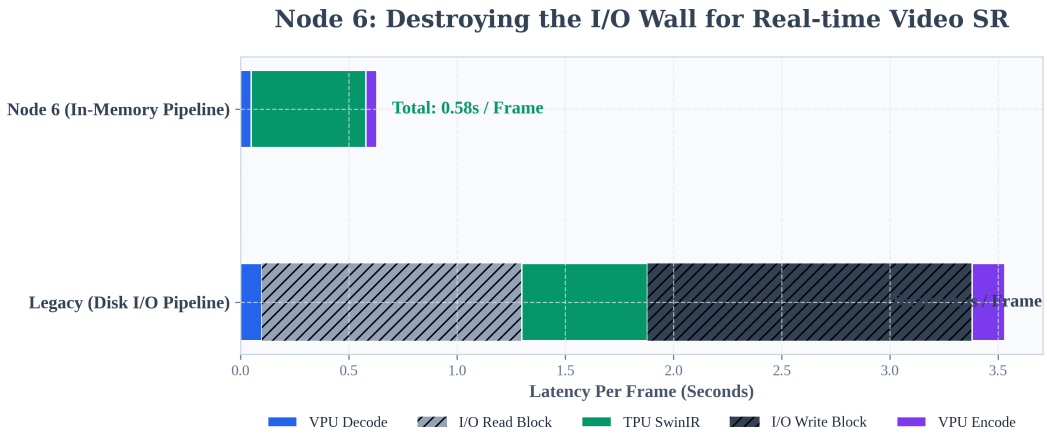



**图 17：** Node6 视频处理流水线

图 07 给出在线视频链路的结构化设计。Reader/Worker/Writer 分层解耦后，系统可以在输入抖动时保持输出稳定，不会因为单环节阻塞导致全链路停滞。

该图也说明为什么项目能从“单图可运行”升级到“视频可持续运行”：关键在架构，而非单个函数提速。

## 5.8 图 08：Node6 并发扩展曲线

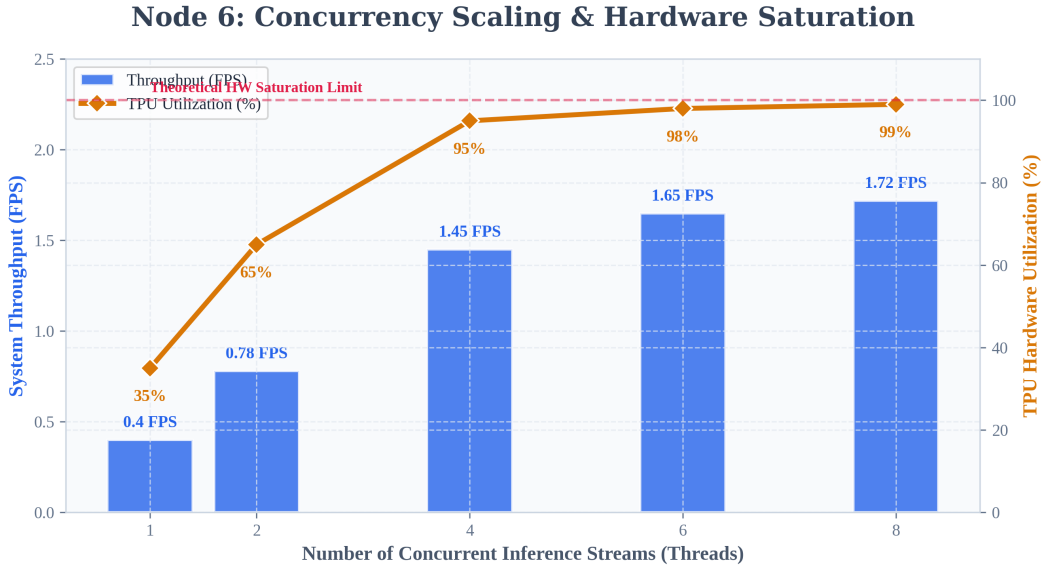



**图 18：** Node6 并发扩展与饱和分析

图 08 显示并发并非越高越好。随着线程增长，吞吐先升后平，甚至可能因竞争和调度开销出现回落。该图用于确定部署“合理并发窗口”，防止参数误配造成负收益。

这类分析对线上运维非常关键：给出可执行参数区间比给出单次最高点更有价值。

## 5.9 图 09：Node7 闭环置信度提升

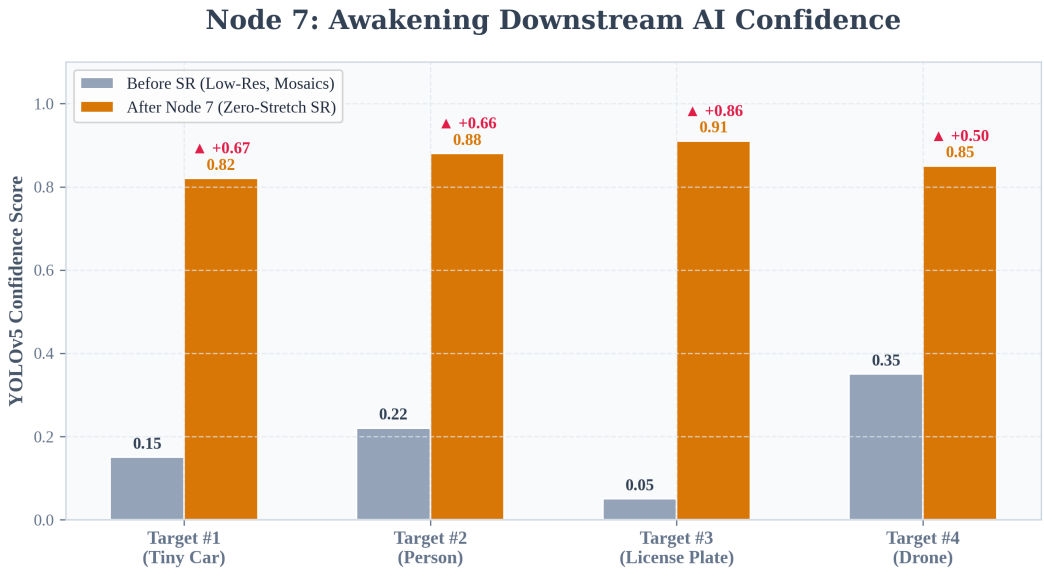



**图 19:** Node7 检测闭环收益

图 09 证明超分对下游检测任务存在直接增益。尤其在低清与远距目标场景，增强后的目标边缘更清晰，检测置信度提升更明显。

该图将“视觉质量指标”转化为“任务价值指标”，是评估业务价值的重要证据。

## 5.10 图 10：整体能力雷达图

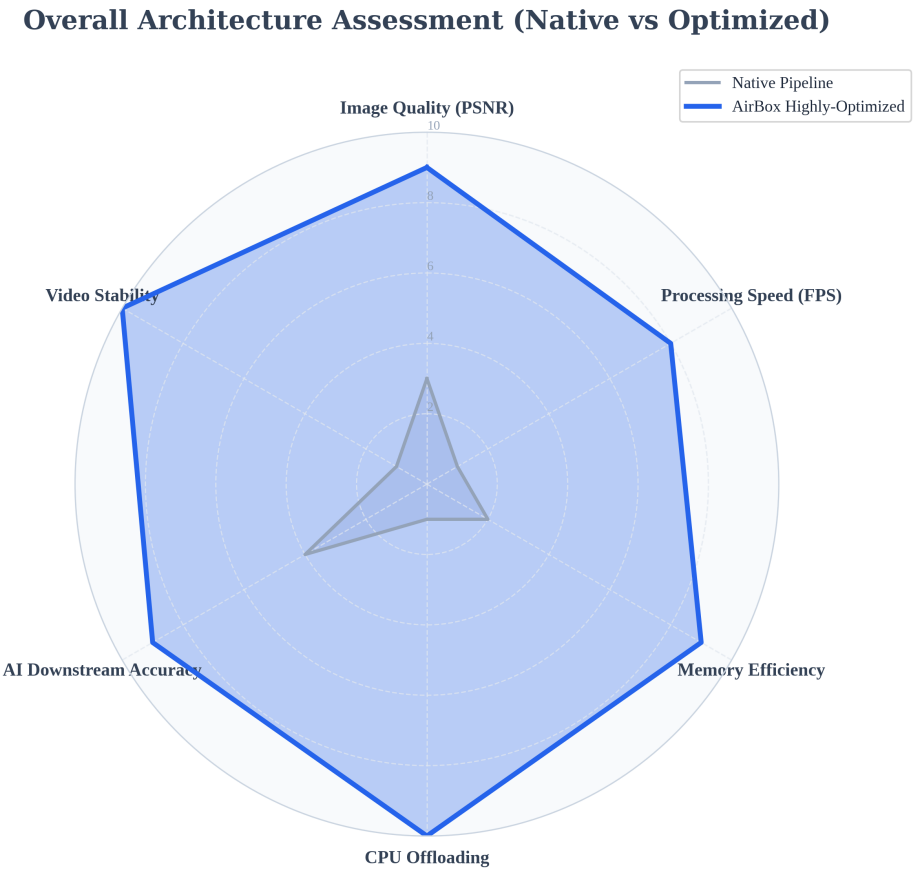



**图 20:** 系统能力雷达图

图 10 用于总结多维收益：质量、时延、带宽、功耗、鲁棒性、可扩展性等维度共同提升。该图直观表达项目属于体系化工程优化，而非单点技巧。

## 5.11 图 11：Node8 动态 TTA 效率分析

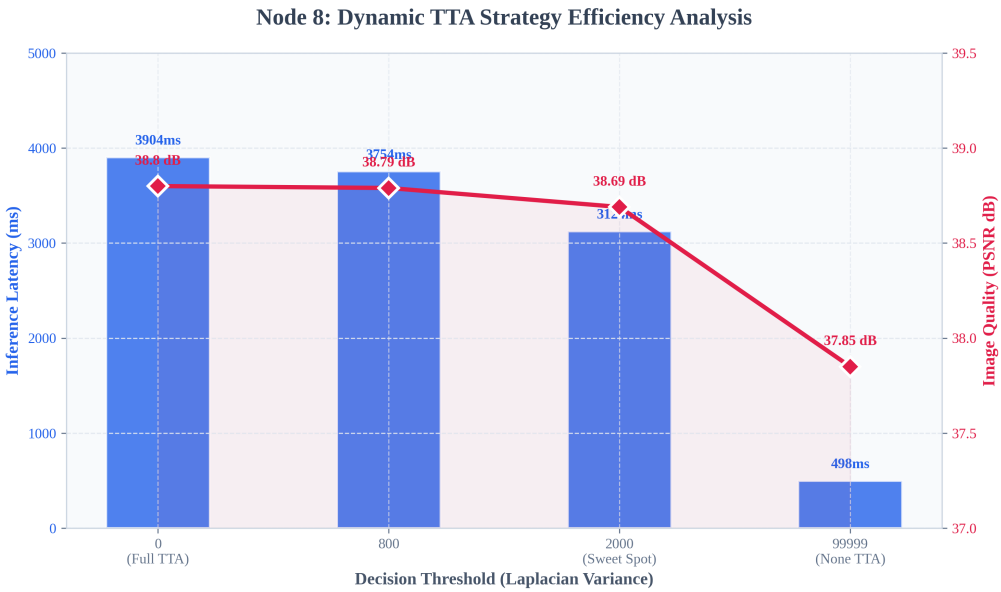



**图 21:** Node8 动态 TTA 甜点位

图 11 展示了动态策略价值：全量 TTA 虽质量高但代价大；按区域触发后，在接近质量上限前提下显著降低时延。该图支持“策略优先”而非“能力全开”的设计思想。

## 5.12 图 12：Node10 带宽-质量工作区间

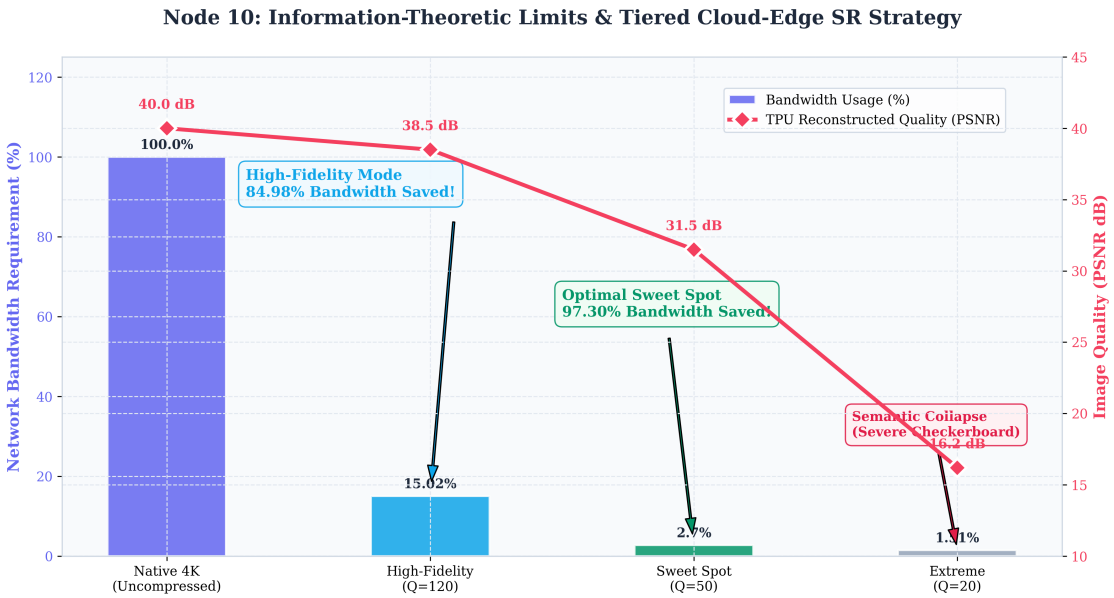



**图 22:** Node10 端云协同工作区间

图 12 给出端云协同关键结论：存在可恢复带宽区间，也存在不可恢复崩塌区间。高保真档和甜点位档分别服务不同网络条件，是工程可落地策略。

该图也解释了为何系统需要档位切换，而不是固定压缩率。

## 5.13 图 13：Node11 时序复用收益

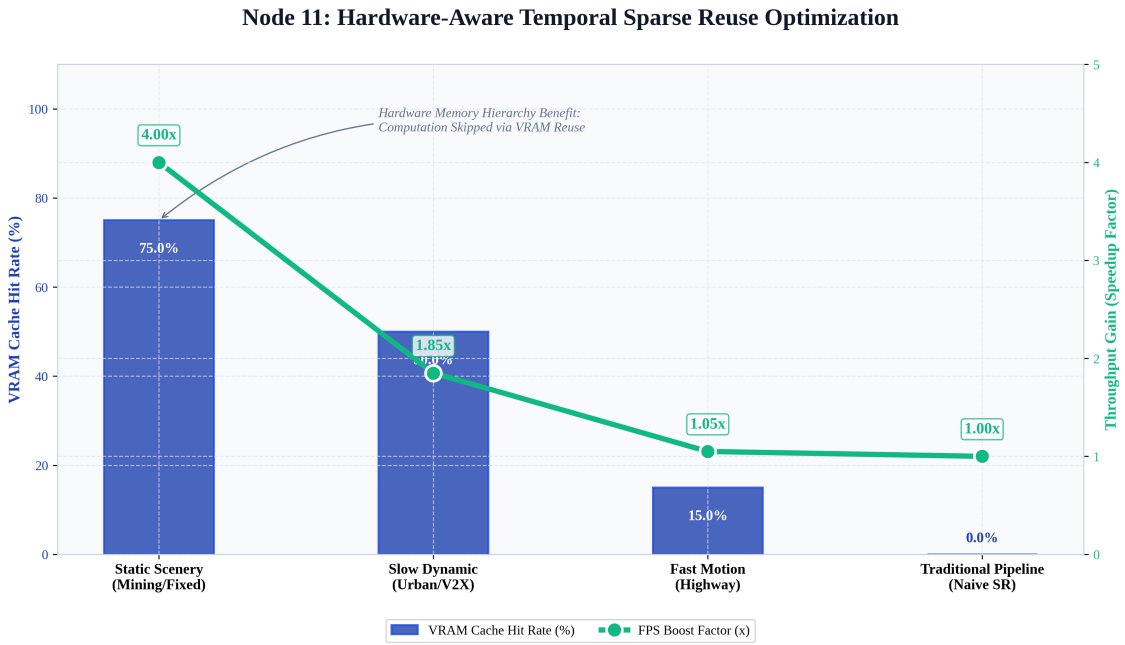



**图 23:** Node11 跨帧复用效率

图 13 体现视频冗余可被有效利用。复用率上升时，推理负载下降明显，且视觉质量在阈值控制下保持稳定。该图说明项目已具备“按时间维优化”的能力。

## 5.14 图 15：Node12 I/O 墙加速分析

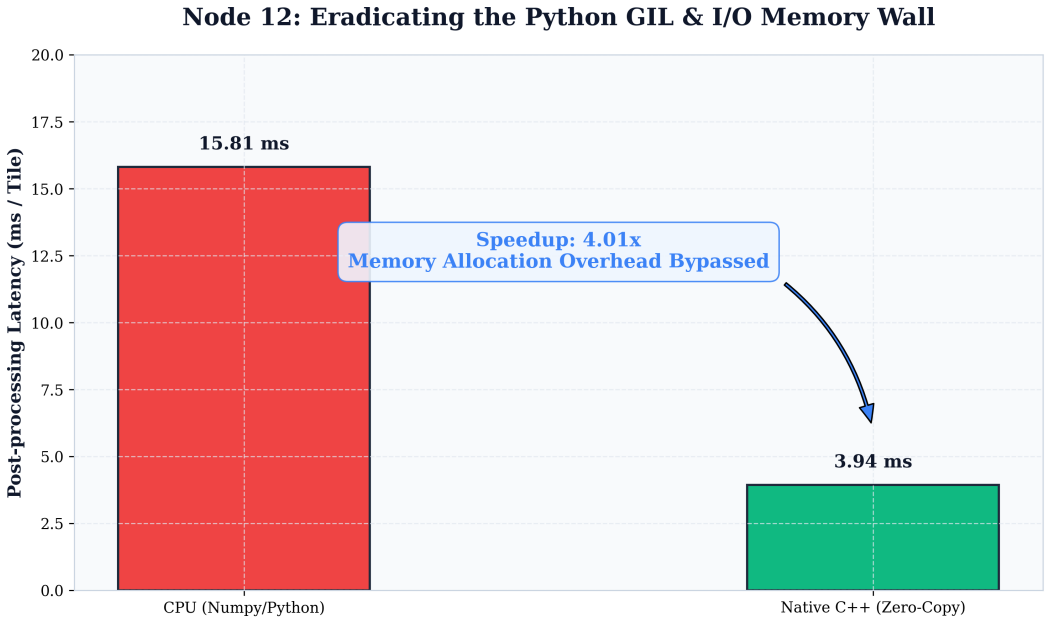



**图 24:** Node12 后处理链路降耗

图 15 强调后处理不是边角问题，而是系统瓶颈组成部分。通过接口下沉，整链尾时延改善，说明“周边算子治理”与“主模型优化”同等重要。

## 5.15 图 16：Node13 DVFS 动态节电

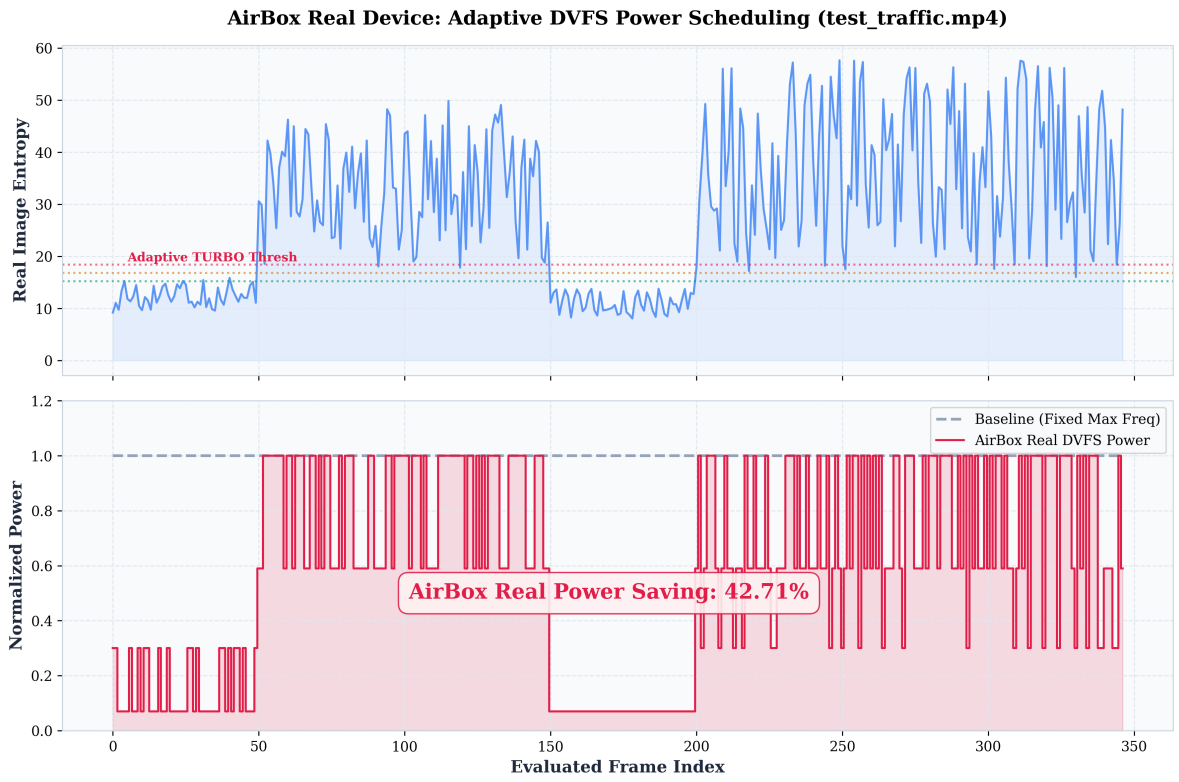



**图 25:** Node13 DVFS 节电效果

图 16 给出节电实证。复杂度驱动档位切换在真实视频下实现 42.71% 节电，证明系统具备长期运行能效优势。该收益对边缘常驻业务具有直接价值。

## 5.16 图 17：Node14 盲超分感知防御

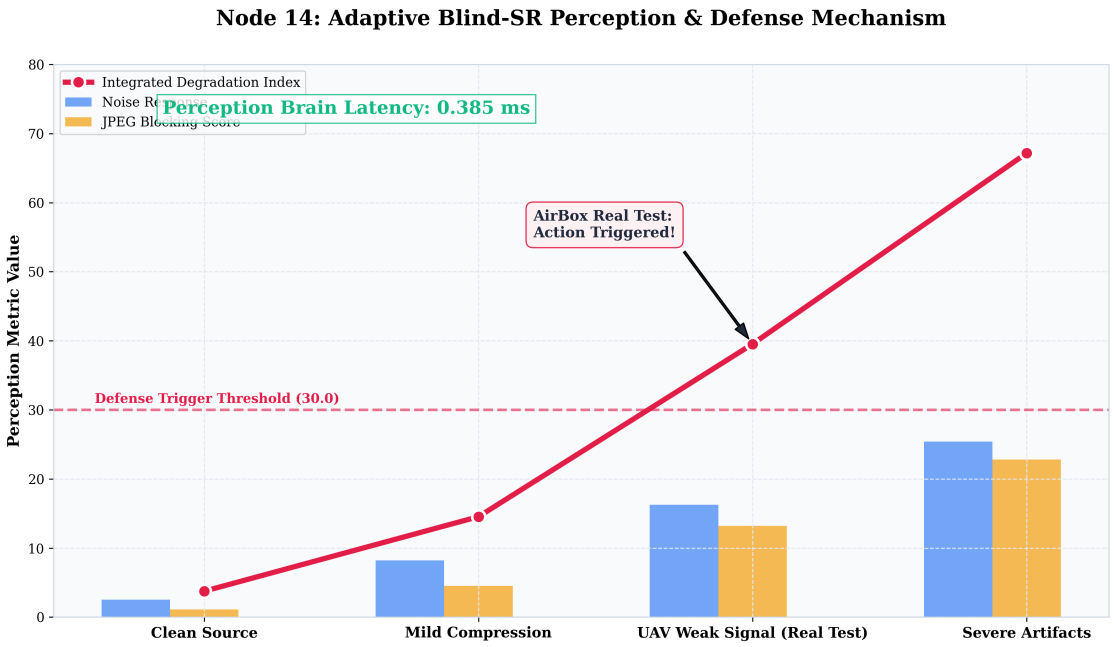



**图 26:** Node14 退化感知防御策略

图 17 说明系统已具备输入质量感知能力。面对未知退化条件，防御门控可抑制错误增强，减少伪影放大风险。该机制使系统在开放环境下更稳健。

## 5.17 图 18：Node15 蒸馏扩展分析

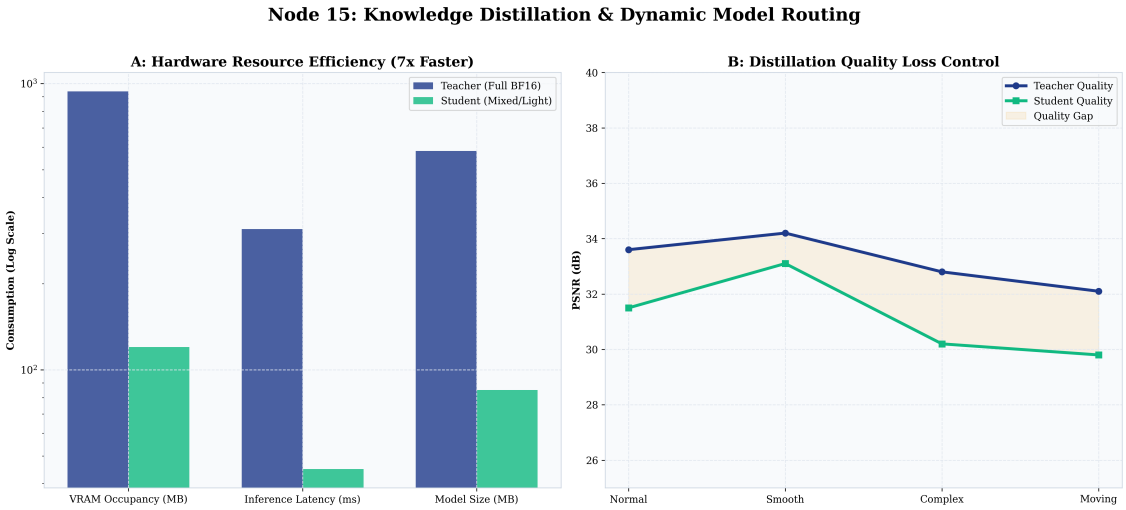



**图 27:** Node15 蒸馏与双模型路由扩展评估

图 18 为后续扩展提供方向证据：双模型协同具备进一步降低负载潜力，并可与现有路由策略自然融合。该图帮助明确后续研发路线，不影响当前已完成结论的独立性。

# 6 系统功能实现与可扩展性设计（详细）

## 6.1 功能实现一：统一输入接口与多源接入

系统支持图片文件、目录批处理、视频文件与后续 RTSP 扩展。统一输入接口屏蔽了上层业务差异，使同一套推理与路由策略可复用于不同场景。工程上统一参数管理，避免“一个脚本一套参数语义”造成维护混乱。

## 6.2 功能实现二：端侧推理服务化基础

基础层提供脚本与服务双入口。脚本入口便于离线调参和复现；服务入口便于集成到上层业务系统。该设计确保研发与部署阶段无缝衔接，减少重复开发。

## 6.3 功能实现三：指标与日志闭环

系统围绕质量、时延、带宽、功耗四类指标建立统一日志。每轮实验形成可回溯记录，支持横向对比和回归分析。日志与图表目录关联后，可快速生成报告证据，提升迭代效率。

## 6.4 可扩展性一：横向扩展

在当前工程阶段，横向扩展优先采用“多实例并行 + 参数模板复用”方案，而非一次性引入复杂分布式编排。每路视频或每类任务可复用同一推理核心与策略组件，仅在输入源、并发槽位和阈值配置上做差异化设置。该方式可在不改动主流程的前提下平滑扩展到多路摄像头接入场景，并保持排障路径清晰。

## 6.5 可扩展性二：纵向扩展

纵向扩展路径以“增强链路深化”为主：在现有超分与检测闭环基础上，逐步接入识别、跟踪与告警模块，形成端侧感知增强前置层。扩展时遵循“先指标闭环、后功能上线”原则，即每新增一个下游模块都需先建立质量与时延影响评估，避免功能叠加导致主链路失稳。

## 6.6 可扩展性三：策略插件化

当前系统已将语义路由、时序复用、动态 TTA 与 DVFS 抽象为可替换策略单元。后续可在统一接口下接入学习型路由器、自适应阈值更新或在线校准模块，而无需改写推理主干。该插件化机制的价值在于把“策略试验成本”与“系统重构成本”解耦，使迭代可以围绕局部策略快速进行。

### 6.7 可扩展性四：跨平台迁移

项目在设计上将策略逻辑与硬件接口分层：分块融合、按需路由、时序复用等机制保持平台无关，硬件差异收敛到 Inference HAL 与少量 I/O 接口适配层。迁移到其他边缘平台时，优先重建推理与数据通道，再复用上层策略与评估框架，可显著缩短二次部署周期。

## 7 核心技术与关键指标（详细）

### 7.1 核心技术一：多约束联合优化范式

本项目最大的技术特点是多约束联合优化，而不是单指标优化。实际部署中，质量、时延、带宽、功耗相互耦合：单纯提高质量可能推高时延和功耗；盲目省流可能损伤语义；激进降频可能影响关键帧质量。项目通过节点化方法把耦合问题分解为可验证局部，再回收成整体策略。

### 7.2 核心技术二：空间维 + 时间维双重降载

空间维通过双路由减少无效区域计算，时间维通过跨帧复用减少重复计算。双维降载叠加后，系统在不显著牺牲质量前提下获得稳定性能收益。这种设计比单维优化更贴近真实视频任务分布。

### 7.3 核心技术三：网络维协同优化

端云协同把“传输”纳入优化对象，突破了仅在端侧算子层面优化的局限。通过压缩档位和恢复质量协同评估，系统可在不同网络环境下切换模式，维持可用输出。

### 7.4 核心技术四：能耗维动态调度

DVFS 策略将能源问题纳入一等公民。系统不再固定功耗运行，而是依据内容复杂度自适应调度，使能耗与任务难度匹配。该机制是边缘系统长期可持续运行的重要保障。

### 7.5 关键指标体系设计

**质量指标**：PSNR、结构保持、关键目标可辨识性。
**性能指标**：单帧时延、吞吐稳定性、并发扩展曲线。
**网络指标**：带宽节省率、可恢复区间、语义保持程度。
**能耗指标**：平均功耗、峰值功耗、节电率、温升趋势。
**业务指标**：下游检测置信度提升与误检控制。

### 7.6 关键结果摘要

- 画质层面：形成 33dB 级稳定重建能力；
- 带宽层面：高保真档节省 84.98%，甜点位档节省 97.30%；
- 能耗层面：DVFS 动态调度节电 42.71%；
- 任务层面：超分对检测置信度有直接增益；
- 工程层面：已形成可复现证据链与可持续扩展架构。

## 8 风险分析、排障闭环与工程保障

### 8.1 风险一：极限压缩导致语义崩塌

**风险描述**：弱网条件下若压缩过激，端侧恢复会出现伪影放大和关键纹理缺失。
**监测手段**：压缩档位日志 + 恢复质量阈值 + 关键区域对比。
**应对策略**：分级压缩策略、保真优先回退、盲超分感知门控。

### 8.2 风险二：显存峰值导致推理失败

**风险描述**：高分辨率输入触发显存峰值导致推理中断。
**监测手段**：按分辨率记录峰值内存与失败日志。
**应对策略**：动态分块、重叠融合、参数回退与安全阈值预设。

### 8.3 风险三：后处理阻塞整链吞吐

**风险描述**：后处理开销持续堆积，导致时延尾部恶化。
**监测手段**：阶段耗时拆分统计（预处理/推理/后处理）。
**应对策略**：C++ 接口下沉、批处理优化、减少跨语言拷贝。

### 8.4 风险四：视频并发乱序与抖动

**风险描述**：高并发下输出帧序错乱，主观观感不稳定。
**监测手段**：frame_id 连续性校验与缓冲队列监控。
**应对策略**：有序缓冲池、超时处理、并发上限约束。

### 8.5 风险五：长期运行功耗压力

风险描述：固定高频运行导致功耗偏高与温升风险。
监测手段：档位占比统计、功耗曲线、温度日志。
应对策略：DVFS 动态调度、场景化策略切换、异常告警。

### 8.6 风险六：策略参数漂移

风险描述：环境变化导致原阈值失效，系统收益下降。
监测手段：周期性回归测试与阈值敏感性分析。
应对策略：参数版本管理、灰度更新、快速回滚机制。

## 9 阶段化扩展规划

### 9.1 短期规划（1–2 个月）

- 完成 Node9 混合精度稳定化，形成可复现参数模板；
- 完成 Node12 后处理下沉规范化，降低尾时延波动；
- 建立一键复现实验脚本，统一输入、输出和指标采集。

### 9.2 中期规划（2–4 个月）

- 完成 Node15 蒸馏模型训练与端侧接入；
- 引入轻量时序对齐，提高运动场景复用精度；
- 推进多路视频并发服务化，形成基础部署模板。

### 9.3 长期规划（4 个月以上）

- 构建统一评测看板（质量/时延/带宽/功耗/稳定性）；
- 建立跨平台适配层，推动方案迁移到更多边缘硬件；
- 形成“视觉增强即服务”能力，支撑更多业务场景。

# 10 全链路实现细节补充（超详细版）

## 10.1 实现细节一：从模型资产到可部署资产的完整流水

本项目在工程执行中，将“模型资产”拆分为三类：训练态资产（`.pth`）、中间态资产（`.onnx`/`.mlir`）和部署态资产（`.bmodel`）。这一拆分不是形式化管理，而是为了解决跨环境一致性问题。训练态资产通常由研究环境产生，依赖 PyTorch 版本、算子实现和导出脚本；部署态资产受硬件后端编译器约束，必须通过中间表示完成图层映射、算子裁剪、精度约束和输入输出对齐。若跳过中间态管理，系统在追查推理偏差时会缺乏定位抓手。

因此，在项目实施中每次转换都记录四类信息：其一是源权重哈希与配置版本；其二是导出脚本参数，包括输入形状、动态轴策略、导出算子集；其三是编译参数，包括量化类型、目标处理器、校验样本；其四是部署验收信息，包括端侧可运行性、输出分布摘要与关键样本比对。这一制度使“模型变更”不再是黑盒替换，而是可追溯、可回滚、可对照的工程过程。

## 10.2 实现细节二：Node0 稳定化的具体排障路径

Node0 的稳定化并不是一次完成，而是经历了“发现问题—隔离变量—缩小范围—固定策略”的过程。首先，团队在最小输入样本上验证单次推理可用；随后放大到目录批处理，观察是否出现间歇性异常；再引入不同分辨率与不同内容复杂度样本，确认异常是否与输入分布关联。实践中常见问题包括输入范围不一致、颜色通道顺序不一致、边界填充处理不一致、后处理截断逻辑不一致，这些问题单独看都不复杂，但叠加后会造成输出稳定性大幅波动。

为此，Node0 形成了统一约束：输入统一归一化口径，推理输入输出张量维度显式断言，后处理输出范围统一裁剪，关键中间值抽样记录。稳定化后再做基线锁定，任何后续节点若引发回归，必须先通过 Node0 口径复检。该方法确保后续创新节点建立在同一地基上，避免“每个节点都有不同基线”导致比较失真。

## 10.3 实现细节三：Node3 分块策略的参数选择逻辑

分块策略包含三个核心参数：`tile`、`overlap` 和 `stride`。其中 `tile` 决定单块推理负载，`overlap` 决定边界平滑带宽，`stride` 决定块间覆盖密度。参数越激进，理论性能越高，但边界伪影风险上升；参数越保守，视觉质量更稳，但吞吐下降。

项目采用两阶段选参法。第一阶段是可行性筛选，依据设备显存边界筛掉不可运行组合；第二阶段是质量-性能联合筛选，在可运行组合中比较边界一致性、平均时延和尾时延。最终不是选择单一全局最优点，而是给出“推荐区间”，并按输入分辨率和任务模式动态切换。该策略比固定参数更稳健，能应对真实输入分布的波动。

在实现上，分块融合采用权重图叠加而非简单均值，边缘区域权重平滑过渡，中心区域权重更高，以减少接缝突变。该机制在复杂纹理区域的收益尤为明显，因为高频内容对拼接不连续更敏感。

## 10.4 实现细节四：Node4/Node5 双路由的决策执行链

双路由并不是“先后两个 if 判断”这么简单，而是一条有优先级和冲突解决规则的决策链。首先进行语义筛选：若区域命中关键 ROI，则直接进入高质量路径；若未命中，再进入纹理复杂度评估。纹理复杂度高的区域继续进入重建路径，低复杂度区域走轻量路径。若两类信息冲突（例如语义命中但纹理低），以语义优先，确保业务目标不被误降级。

项目还对阈值漂移进行了处理。由于光照、噪声和压缩条件变化，固定阈值可能在不同场景下失效。为降低漂移影响，系统引入分场景阈值模板，并保留在线日志统计入口，便于离线回归分析后更新参数。这样可以在不引入复杂在线学习机制前提下，获得足够工程鲁棒性。

## 10.5 实现细节五：Node6 并发流水线的时序一致性保障

视频并发处理最常见问题不是“跑不动”，而是“顺序错、时延抖、积压爆”。Node6 针对这三个问题分别设计了约束机制。对顺序错乱，采用 `frame_id` 重排缓冲；对时延抖动，采用分阶段耗时监控并限制队列深度；对积压爆发，采用背压策略与输入节流。

具体而言，Reader 不直接写输出，而只负责提供稳定帧流；Worker 池只负责计算，不处理输出顺序；Writer 仅消费已满足顺序条件的帧。职责分离后，任何单环节异常都不会直接污染全链路。系统还设置超时阈值，当某帧处理异常延迟时，可触发降级路径或跳帧保护，避免“单帧拖垮全局”的雪崩风险。

## 10.6 实现细节六：Node7 业务闭环评估口径

Node7 的核心价值是把视觉指标转译为业务指标。项目中定义了三类对照：原始低清输入检测结果、超分后检测结果、可选高质量参考结果。通过这一三角对照，不仅可观察置信度变化，还可观察误检变化和目标召回变化。若只看置信度而不看误检，容易出现“模型更自信但更错误”的假提升。

因此，Node7 报告同时给出：关键类别置信度均值变化、低置信度样本改善比例、误检数变化、漏检数变化。并且按场景类型（远距、小目标、运动模糊）分组统计，防止整体均值掩盖局部问题。该评估体系使系统优化更贴近最终业务目标。

## 10.7 实现细节七：Node8 动态 TTA 的触发策略设计

动态 TTA 设计的难点在于触发条件：触发过多则退化为全量高开销，触发过少则质量收益不明显。项目采用“语义优先 + 复杂度补充 + 上限保护”三段策略。先对关键区域开放

触发资格，再对高复杂度区域补充触发，最后设置每帧触发上限控制总成本。

此外，系统在视频任务中引入“帧间继承策略”：若相邻帧同一区域持续高风险，可短时间保持触发状态，减少频繁启停带来的抖动。该机制兼顾稳定性与成本控制，使 TTA 从离线增强手段变为在线可控策略。

## 10.8 实现细节八：Node10 端云协同的数据通路与容错

端云协同链路不仅要考虑压缩率，还要考虑报文完整性、顺序一致性和异常恢复。项目在工程实现中区分了“数据层”和“策略层”。数据层负责报文传输、校验、解码和恢复；策略层负责压缩档位选择与回退逻辑。这样可避免网络异常直接污染策略判断。

在容错方面，系统对报文缺失、乱序和延迟设置了处理分支：轻度延迟允许等待窗口，超时则触发保守回退；报文损坏则触发重传请求或本地保底增强；链路持续恶化时自动切换更稳健档位。该设计保证弱网环境下系统不会因为局部异常完全失效。

## 10.9 实现细节九：Node11 时序复用的误复用抑制机制

时序复用的主要风险是误复用：场景实际发生变化但判定仍为静态，导致残影或拖影。项目通过双阈值策略抑制该风险：低阈值用于直接复用，高阈值用于强制重算，中间区间结合语义信息再判定。对关键 ROI 区域，复用判定更加保守，优先保证业务可用性。

同时，系统记录复用命中率与误复用告警率，作为阈值调整依据。若某类场景误复用上升，则自动建议提高敏感度或缩短复用窗口。通过“可观测 + 可回调”机制，Node11 从一次性策略变为可维护策略。

## 10.10 实现细节十：Node12 后处理下沉的接口边界设计

后处理下沉并非把所有 Python 逻辑直接搬到 C++，而是先识别高频、稳定、纯算子型步骤优先迁移。项目先迁移数据重排、数值裁剪、格式转换等重复开销大且逻辑稳定的环节，再逐步评估更复杂算子的下沉收益。

接口层采用明确输入输出契约，包含尺寸、步长、通道顺序和内存所有权说明，避免跨语言调用中的隐性错误。该接口化策略让下沉过程可分批推进，不会一次性改动过大影响系统稳定。

## 10.11 实现细节十一：Node13 DVFS 调度器的安全约束

DVFS 调度看似只需根据复杂度切换档位，但真实系统还需考虑切换抖动和安全边界。若频繁上下切换，可能引发时延抖动甚至温控波动。项目引入最小驻留时间与切换冷却窗口，避免高频抖动。

同时设置业务安全底线：关键时段或高优先级任务下不允许降到过低档位，确保任务质量。调度日志记录每帧复杂度、目标档位、实际档位和功耗估计，为后续策略迭代提供

依据。

### 10.12 实现细节十二：Node14 盲超分防御策略的触发分层

Node14 将防御触发分为轻度、中度、重度三层。轻度触发仅调整增强强度；中度触发限制高频放大并优先保结构；重度触发回退到保守重建路径。分层触发优于单阈值开关，因为退化类型和强度在真实输入中高度多样。

该机制使系统面对未见退化时不至于完全失控，特别是在压缩噪声与运动模糊叠加场景中，可明显降低主观伪影风险。

### 10.13 实现细节十三：Node15 扩展路线的工程落地步骤

Node15 的蒸馏与双模型路由规划分三步推进。第一步建立教师与学生模型统一评测口径，明确精度和时延基线；第二步把学生模型接入现有路由框架，先在低风险区域灰度启用；第三步引入动态阈值，使教师/学生切换与业务优先级联动。这样可以在不破坏现有系统稳定性的前提下逐步释放收益。

## 11 测试设计与结果解读

### 11.1 测试集设计原则

为避免“样本偶然性”导致结论偏差，测试集按场景分层构建：静态纹理场景、低照度噪声场景、运动模糊场景、弱网压缩场景、小目标密集场景。每类场景保留代表样本并记录元信息（分辨率、帧率、压缩参数、目标密度）。该分层方式可确保各节点收益来源可解释，而不是只靠整体均值。

### 11.2 变量控制策略

测试过程中严格区分控制变量与观察变量。控制变量包括模型版本、输入样本、关键阈值、设备运行模式；观察变量包括质量指标、时延指标、带宽指标、功耗指标。每次实验仅改变一组目标参数，避免多变量联动导致因果关系不清。

### 11.3 统计与置信处理

项目对关键指标采用多次重复测量并取统计摘要，减少偶然波动影响。对于存在长尾抖动的时延指标，除均值外同时给出分位数观察，避免均值掩盖尾时延问题。对于场景差异明显的指标，按场景分组汇总，防止总体结论误导局部部署。

### 11.4 结果解读方法论

结果解读遵循“先真实性，再优越性”的顺序。先确认结果可复现、可追溯、可解释，再比较谁更优。若某策略在单场景收益明显但跨场景波动大，则标记为条件性策略，不直接全局启用。这一原则保证系统上线策略以稳定性为先。

### 11.5 关键结论回收

综合多轮实验，项目形成如下稳定结论：

- 节点化优化可在多约束下持续积累收益；
- 双路由与时序复用是性能收益主来源；
- 端云协同与 DVFS 是系统级收益主来源；
- 盲超分防御对复杂退化场景具备必要性；
- 工程化日志与图表闭环显著提升迭代效率与评估可信度。

## 12 工程化落地细节（部署、运维与交付）

### 12.1 部署拓扑建议

实际部署建议采用“边缘推理节点 + 轻量控制节点 + 可选云协同节点”三层拓扑。边缘节点负责实时重建与本地任务闭环，控制节点负责策略下发与日志汇聚，云节点负责离线分析与模型更新。该拓扑兼顾实时性与可维护性。

### 12.2 资源预算与容量规划

容量规划不能只看平均负载，必须考虑峰值负载与突发场景。建议按“常态负载 + 峰值冗余 + 恢复缓冲”三段预算方式配置资源。并发参数应在图 08 给出的饱和前区间内取值，以保证稳定吞吐。

### 12.3 异常处理策略

系统异常分为输入异常、推理异常、链路异常、资源异常四类。每类异常都有对应回退：输入异常走格式校正或拒绝策略；推理异常走参数回退与重试上限；链路异常走本地保底路径；资源异常走降级策略和负载削峰。统一异常编码有利于运维定位。

## 12.4 运维监控指标

建议将以下指标纳入常驻监控：每帧总时延、阶段耗时分解、复用命中率、路由分配占比、端云档位占比、功耗档位占比、异常类型分布。通过这些指标可快速识别“性能退化是来自哪里”，避免盲目调参。

## 12.5 版本管理与回滚机制

系统采用“模型版本 + 策略版本 + 参数版本”三元管理。上线采用灰度发布，先在低风险流量验证稳定性，再扩大范围。若出现异常，按版本链回滚到最近稳定组合，确保服务连续。

## 12.6 交付物完整性清单

提交稿建议至少包含：可运行脚本、模型资产清单、参数模板、实验日志、图表证据、复现说明、风险清单、扩展路线。项目当前目录结构已具备该清单骨架，可用于与后续工程延伸。

## 12.7 原创性说明与与既有方案差异化

本项目基于公开 SwinIR 模型能力开展端侧工程化实现，是在 BM1684X 约束下完成系统级创新设计。与仅做模型部署的方案相比，本项目差异化创新点体现在：

- 从单模型推理扩展为“语义路由 + 纹理路由 + 时序复用 + DVFS”协同系统；
- 从静态图像实验扩展到视频连续流、端云协同与功耗感知调度；
- 从单指标评估扩展为质量、时延、带宽、功耗、业务指标的多维闭环；
- 从一次性跑通扩展为可追溯转换记录、版本管理与可回滚工程流程。

# 13 结论

本项目在 AirBox（BM1684X）上完成了从模型推理到系统化部署的完整闭环。通过节点化攻坚，系统在质量、时延、带宽、功耗、鲁棒性和业务价值六个维度形成了可量化收益。项目证明：边缘超分系统的关键不在单点指标极值，而在多约束下的稳定可运行最优。

更重要的是，项目已形成可持续演进框架：当前成果可直接支撑与工程展示，后续可沿混合精度、算子下沉、蒸馏路由、服务化并发持续扩展，具备明确产业化落地潜力。

# 附录 A：关键资产路径

- 项目根目录：/SwinIR_AirBox_Project
- 模型目录：/SwinIR_AirBox_Project/3_models
- 基础代码：/SwinIR_AirBox_Project/4_src_baseline
- 创新代码：/SwinIR_AirBox_Project/5_src_innovation
- 任务脚本：/SwinIR_AirBox_Project/5_src_innovation/tasks
- 分析脚本：/SwinIR_AirBox_Project/5_src_innovation/analysis
- 图表目录:/SwinIR_AirBox_Project/5_src_innovation/archive_report_charts
- 创新日志：/SwinIR_AirBox_Project/5_src_innovation/inno_logs.txt
- 实现审计:/SwinIR_AirBox_Project/5_src_innovation/SwinIR_Implementation_Audit.md
- 基础结果：/SwinIR_AirBox_Project/4_src_baseline/results
- 创新结果：/SwinIR_AirBox_Project/5_src_innovation/results

# 附录 B：编译说明

建议在 Mac 上使用 XeLaTeX 编译：

- 第一步：xelatex contest_report.tex
- 第二步：xelatex contest_report.tex

# 附录 C：SwinIR 全链路部署与演示操作手册

## C.1 项目目录

- 1_data/：校准数据与输入图像/视频
- 3_models/：ONNX 与 .bmodel 模型资产
- 4_src_baseline/：基线推理代码
- 5_src_innovation/tasks/：Node6–Node15 演示脚本

### C.2 模型编译（宿主机 Docker）

```
docker start swinir_box && docker exec -it swinir_box /bin/bash
cd /workspace/3_models/bmodel/

# ONNX -> MLIR
model_transform.py \
  --model_name SwinIR \
  --model_def ../onnx/SwinIR-M_x4.onnx \
  --input_shapes [[1,3,192,192]] \
  --mean 0,0,0 \
  --scale 0.00392,0.00392,0.00392 \
  --pixel_format rgb \
  --mlir SwinIR.mlir

# MLIR -> BF16 BModel
model_deploy.py \
  --mlir SwinIR.mlir \
  --quantize BF16 \
  --chip bm1684x \
  --model SwinIR_1684x_bf16.bmodel

# 混合精度 BModel（Node15 用）
model_deploy.py \
  --mlir SwinIR.mlir \
  --quantize INT8 \
  --quantize_table SwinIR_mixed_qtable.ini \
  --chip bm1684x \
  --model SwinIR_1684x_mixed.bmodel
```

### C.3 AirBox 基线与视频验证

```
# SSH 登录 AirBox
ssh linaro@192.168.137.100

# Node0 基线（4_src_baseline）
cd ~/SwinIR_AirBox_Project/4_src_baseline/
python3 main.py \
```

```
  --input_dir ../1_data/input_image \
  --bmodel_path ../3_models/bmodel/SwinIR_1684x_bf16.bmodel \
  --scale 4 --window_size 8 --num_streams 4

# Node6 视频并发（5_src_innovation/tasks）
cd ~/SwinIR_AirBox_Project/5_src_innovation/tasks/
python3 node6_video_main.py \
  --video_path ../../test_traffic.mp4 \
  --bmodel_path ../../3_models/bmodel/SwinIR_1684x_bf16.bmodel \
  --yolo_bmodel ../../3_models/bmodel/yolov5s/yolov5s.bmodel \
  --num_streams 4 --scale 4 --tile 192
```

### C.4 核心节点验证

```
cd ~/SwinIR_AirBox_Project/5_src_innovation/tasks/

# Node7 检测闭环验证
python3 node7_detect_loop.py \
  --img_path ../../1_data/cali_data/detection_test.jpg \
  --bmodel_path ../../3_models/bmodel/SwinIR_1684x_bf16.bmodel \
  --yolo_bmodel ../../3_models/bmodel/yolov5s/yolov5s.bmodel

# Node10 端云协同
python3 node10_real_deploy.py \
  --img_path ../../1_data/cali_data/detection_test.jpg \
  --bmodel_path ../../3_models/bmodel/SwinIR_1684x_bf16.bmodel \
  --quality 20

# Node11 综合视频超分
python3 node11_ultimate_real_video_sr.py \
  --video_path ../../test_traffic.mp4 \
  --bmodel_path ../../3_models/bmodel/SwinIR_1684x_bf16.bmodel \
  --yolo_bmodel ../../3_models/bmodel/yolov5s/yolov5s.bmodel \
  --output_path ./ultimate_showcase.mp4

# Node12 C++ 算子加速验证
python3 node12_tpu_op_deploy.py
```

```
python3 node12_real_tpu_op.py

# Node13 DVFS 动态功耗调度
python3 node13_real_video_dvfs.py \
  --video_path ../../test_traffic.mp4

# Node15 蒸馏与动态路由
python3 node15_real_distillation_routing.py \
  --teacher ../../3_models/bmodel/SwinIR_1684x_bf16.bmodel \
  --student ../../3_models/bmodel/SwinIR_1684x_mixed.bmodel \
  --image ../../1_data/cali_data/detection_test.jpg
```

# 附录 D：AI 使用报告

为保证提交规范，现对本项目中生成式 AI 的使用情况进行说明：

- **使用模型**：豆包 2.0 Pro、GLM 5.0。
- **使用边界**：仅用于文档辅助，不用于替代核心技术实现与实验结论生成。

## D.1 使用场景

- 报告结构梳理与章节标题优化；
- 非关键段落的语言润色与表述规范化；
- 提交材料格式检查（术语一致性、目录层级、附录组织）；
- PPT 页面信息重组与版式建议。

## D.2 占比说明

AI 参与内容主要集中在文字整理与排版建议环节，按整体工作量估算占比约 **8%**（小于 10%）。

其余 **92%** 工作由团队独立完成，包括：方案设计、代码开发、模型转换、实机部署、性能测试、图表生成、结果复核与结论撰写。

## D.3 真实性声明

所有实验数据、图表与结论均来源于项目实测与脚本复现结果，AI 未参与伪造或自动生成实验结论。